Physica D 490 (2026) 135174

Contents lists available at ScienceDirect

Physica D

journal homepage: www.elsevier.com/locate/physd

Kernel-based PINNs for time fractional differential equations and Volterra integral-differential equations ⋆

Ruimin Gao [a], Lili Ju [b], Dongfang Li [a,c,*]

[a] School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, 430074, China
[b] Department of Mathematics, University of South Carolina, Columbia, SC, 29208, USA
[c] Hubei Key Laboratory of Engineering Modeling and Scientific Computing, Huazhong University of Science and Technology, Wuhan, 430074, China

ARTICLE INFO

Editor: Dmitry Pelinovsky

Keywords:
Physics-informed neural networks
Volterra integral equations
Time fractional differential equations
Sum-of-exponentials approximation
Error estimates

ABSTRACT

In this paper, we propose a novel kernel-based physics informed neural networks (kPINNs) framework for solving time fractional differential equations and Volterra integral differential equations. The highlights mainly consist of two aspects. On one hand, automatic differentiation can be applied more directly and effectively by the present framework. In contrast, the previous neural network approaches for such problems typically depend on difference discretization of the integral operator. On the other hand, to overcome the difficulties from the stiffness, the weights are incorporated into the loss functions. This allows the kPINNs to handle the stiff systems effectively. And the kPINNs can reduce computational cost and improve accuracy for high-dimensional problems. Comparative studies with existing neural network solvers and extensive numerical experiments are also presented to demonstrate the effectiveness and efficiency of our method.

1. Introduction

In this paper, we consider numerical solutions of the Volterra integral-differential equations with convolution kernels [1]:

$$\frac{d^n u(\mathbf{x},t)}{dt^n} + \int_0^t k(t-s)\mathcal{A}u(\mathbf{x},s)ds = f(u(\mathbf{x},t),\mathbf{x},t), \quad (\mathbf{x},t)\in\Omega\times(0,T], \tag{1.1}$$

where $n\in\mathbb{N}$ and the time fractional differential equations [2]:

$$\partial_t^\alpha u(\mathbf{x},t) + \mathcal{A}u(\mathbf{x},t) = f(u(\mathbf{x},t),\mathbf{x},t), \quad (\mathbf{x},t)\in\Omega\times(0,T], \tag{1.2}$$

where ∂_t^α denotes the Caputo fractional derivative of order $\alpha\in(n,n+1)$ for $n\in\mathbb{N}$, that is,

$$\partial_t^\alpha u(\mathbf{x},t) = \int_0^t \omega_{n+1-\alpha}(t-r)\frac{\partial^{n+1}u(\mathbf{x},r)}{\partial r^{n+1}}dr, \quad \omega_{n+1-\alpha}(t) = \frac{t^{n-\alpha}}{\Gamma(n+1-\alpha)},$$

with $\Gamma(\cdot)$ denoting the standard Gamma function. The set Ω is an open bounded domain in $\mathbb{R}^d$ with $d\geq 1$, and $\partial\Omega$ is assumed to be sufficiently smooth. The operator $\mathcal{A}$ denotes a strongly elliptic second order differential operator,

$$\mathcal{A}u(\mathbf{x},t) = -\sum_{i=1}^{d}\partial_{x_i}(a(\mathbf{x})\partial_{x_i}u(\mathbf{x},t)) + c(\mathbf{x})u(\mathbf{x},t),$$

where $0<c_1\leq a(\mathbf{x})\leq c_2$, $c(\mathbf{x})\geq 0$, and c_1, c_2 are constants. Here, x_i represents the ith component of $\mathbf{x}$. The Caputo fractional derivative allows the use of classical initial conditions, which have clear physical interpretations. The kernel also makes analytical and numerical treatment more straightforward when memory effects with power-law decay are sufficient. Such nonlocal equations provide a powerful framework for modeling a wide range of natural phenomena in physics, industry, and medical applications [3–6].

Many numerical methods have been developed for solving nonclassical differential equations, particularly for time fractional differential equations (fPDEs). Classical methods include the L1 scheme [7–9], convolution quadrature type methods [10,11], nonuniform $L2$ type schemes [12,13], and so on [14,15]. Due to the nonlocal nature of fPDEs [16–19], many of these approaches require substantial memory, especially for the evaluation of global convolution integrals. To alleviate storage requirements, sum of exponentials (SoEs) approximations have been widely adopted [20–22]. For example, Jiang et al. [23] introduced the SoEs algorithm combined with finite difference methods to solve time fractional diffusion equations and provided both stability and error analyses. Liu et al. [24] proposed a fast scheme for nonlinear subdiffusion equations by discretizing the spatial variables with Galerkin finite element methods and accelerating the evaluation of Caputo derivatives through SoEs approximation. Yuan et al. [25] established a fast L2-1_σ scheme that employed SoEs approximations and Fourier spectral methods to solve time-space fractional Schrödinger equations, and

⋆ The research was supported by the National Natural Science Foundation of China under grants 12571426 and 12231003.
* Corresponding author.
E-mail addresses: ruimingao@hust.edu.cn (R. Gao), ju@math.sc.edu (L. Ju), dfli@hust.edu.cn (D. Li).

https://doi.org/10.1016/j.physd.2026.135174
Received 10 December 2025; Received in revised form 7 February 2026; Accepted 25 February 2026
Available online 28 February 2026
0167-2789/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

derived optimal error bounds. Other numerical methods for solving integro differential equations, we refer to [5,26,27]. Generally speaking, these classical numerical methods are difficult to efficiently solve high-dimensional problems.

Neural networks based machine learning has recently emerged as a powerful tool in scientific computing. These methods can be broadly categorized into two paradigms. The first is neural operators, which learn solution operators of partial differential equations (PDEs), such as DeepONet [28–30], Fourier neural operators [31,32], and graph neural operators [33]. The second includes neural networks that approximate mappings from the space time domain to solution functions, such as the neural Galerkin scheme [34,35], PINNs [36–39], and related optimization algorithms [40–44], see also [45–47]. In particular, PINNs have been successfully applied to many PDEs, including high-dimensional PDEs [48–51], stochastic differential equations [52], and various other problems [53–55].

PINN-based machine learning methods have also been extended to fPDEs. For example, Wang et al. [56] developed a novel PINN method for variable order space fractional advection diffusion equations. They reduced training time by adjusting the learning rates based on error analysis. Pang et al. [57] introduced fractional PINNs (fPINNs) for solving space time fractional advection diffusion equations. To minimize the loss function, they analyzed the relation between discretization points and sample errors. Up to now, fPINNs have been widely used in solving several forward and inverse problems [58–60]. Besides, Lu et al. [61] proposed an fPINN deeponet framework that learns integrated operators for fPDEs. Guo et al. [62] established Monte Carlo fPINNs (MC-PINNs) to solve high-dimensional fPDEs, where the Monte Carlo method is used to approximate integral kernels. They also obtained an unbiased estimate of physical constraints in the loss function. Wang and Karniadakis [63] further extended this method to general MC-PINNs for solving fPDEs on irregular domains. Generally speaking, the existing neural network solvers are developed by the approximation of the nonlocal operator. They require the storage of the solutions of fPDEs at all previous time steps. It leads to a considerable computational cost and storage in the long-time simulations of these nonlocal problems.

In this paper, we propose a novel neural network framework for solving Volterra integral differential equations and time fractional differential equations. The main contributions are summarized as follows:

- The novel framework is developed by combining the SoEs approximations and the PINNs. Within this framework, automatic differentiation techniques can be applied directly and effectively to non-classical differential equations, in contrast to previous neural network approaches that typically depend on difference discretization of nonlocal operators.
- Using SoEs approximations, nonlocal equations are transformed into a system of integer order differential equations. It is shown that the errors between the two systems can be sufficiently small and are controlled by the error of the SoEs approximation.
- The resulting local system consists of families of integer order differential equations with varying stiff terms. To overcome the difficulties, weights w_i are incorporated into the loss functions. These super parameters, aligned with the SoEs approximations, allow the proposed kernel-based PINNs (kPINNs) to handle stiff systems effectively. Consequently, PINNs can perform well even for stiff classic differential equations.
- Comparative numerical studies are presented with existing neural network solvers, and extensive numerical experiments demonstrate that the proposed kPINNs are more competitive in terms of efficiency and accuracy.

It is remarkable that the SoEs approximations are widely applied in some classical numerical methods and give some fast schemes [23–25]. The difference schemes are still used in the approximations of the nonlocal operator. In this study, we apply the SoEs approximations and transform the nonlocal problems into a system of integer order differential equations. The interesting transformation offers an advantage that automatic differentiation techniques can be applied directly and efficiently. And the nonlocal problem can be solved step by step. Moreover, the L1-type method requires $\mathcal{O}(N_T N_s)$ memory and $\mathcal{O}(N_T^2 N_s)$ work, where N_T and N_S represent the total number of time steps and spatial sample points in domain [23]. In contract, SoEs approximations show an improvement in reducing memory usage and time complexity with $N_{\mathcal{P}} \sim \mathcal{O}(\log N_T)$, which only requiring $\mathcal{O}(N_s N_{\mathcal{P}})$ storage and $\mathcal{O}(N_s N_T N_{\mathcal{P}})$ work. Therefore, by combining the neural network framework and SoEs approximation, the present kPINNs are much more effective than the existing methods in solving the high-dimensional time fractional differential equations and Volterra integral-differential equations.

The remainder of the paper is organized as follows. Section 2 presents the application of SoEs approximations to the linear case of the time fractional differential Eq. (1.2) with $\alpha \in (0,1)$. Section 3 introduces the kPINNs for solving the linear time fractional differential equation studied in Section 2, and then discusses its extension and implementation to general time fractional differential equations and Volterra integral-differential equations. Section 4 provides numerical experiments that demonstrate the accuracy and efficiency of kPINNs, and compares their performance with other neural network solvers. Finally, Section 5 concludes the paper with some remarks.

2. Kernel-based approximations

In this section, we present the detailed SoEs approximations and kernel-based transformations. To illustrate the idea more clearly, we first consider the following linear time fractional differential equation:

$$\partial_t^\alpha u(\mathbf{x},t) + \mathcal{A}u(\mathbf{x},t) = f(\mathbf{x},t), \quad (\mathbf{x},t) \in \Omega \times (0,T], \tag{2.1}$$

where $0 < \alpha < 1$.

2.1. Preliminaries

We consider the kernel $k(t)$ to be real valued and positive definite, satisfying [64]

$$k(t) \geq 0, \quad \frac{dk(t)}{dt} \leq 0, \quad \frac{d^2k(t)}{dt^2} \geq 0.$$

To derive a numerical scheme, we apply the SoEs method to approximate the kernel $k(t)$. We define the approximate kernel $\hat{k}(t)$ as

$$\hat{k}(t) = \sum_{i=1}^{N_{\mathcal{P}}} w_i e^{-s_i t}, \tag{2.2}$$

where the approximation satisfies the following property.

Lemma 2.1. *[23] For a given fractional derivative order α, desired precision ϵ, and $0 < \delta \leq t \leq T$, there exists a family of weights w_i and points s_i, $i = 1, \ldots, N_{\mathcal{P}}$ such that*

$$\left| t^{-\alpha} - \sum_{i=1}^{N_{\mathcal{P}}} w_i e^{-s_i t} \right| \leq \epsilon, \quad \forall t \in [\delta, T],$$

where $N_{\mathcal{P}}$ is the number of exponentials required and satisfies

$$N_{\mathcal{P}} = \mathcal{O}\Big((\log(T/\delta) + \log(\log(\epsilon^{-1}))) \log(\epsilon^{-1}) + (\log(\delta^{-1}) + \log(\log(\epsilon^{-1}))) \log(\delta^{-1}) \Big).$$

The weights w_i and the nodes s_i can be determined by the following result.

Lemma 2.2. *[23] Let $\epsilon > 0$ be the desired precision, let $0 < \delta \leq t \leq T$ ($\delta \leq 1$ and $T \geq 1$), and define $p_0 = \mathcal{O}(\log \frac{1}{\epsilon})$, $M = \mathcal{O}(\log T)$, and $N = \mathcal{O}(\log\log \frac{1}{\epsilon} + \log \frac{1}{\delta})$. Furthermore, let $w_{l,1}, \ldots, w_{l,p_0}$ and $s_{l,1}, \ldots, s_{l,p_0}$ denote the weights and nodes for the p_0 point Gauss Jacobi quadrature on $[0, 2^{-M}]$, let $w_{j,1}, \ldots, w_{j,p_1}$ and $s_{j,1}, \ldots, s_{j,p_1}$ denote the weights and nodes for the p_1 point Gauss Legendre quadrature on the intervals $[2^j, 2^{j+1}]$ for*

$j = -M, \dots, -1$, let $p_2 = \mathcal{O}(\log \frac{1}{\epsilon})$, and let $w_{j,1}, \dots, w_{j,p_2}$ and $s_{j,1}, \dots, s_{j,p_2}$ denote the weights and nodes for the p_2 point Gauss Legendre quadrature on the intervals $[2^j, 2^{j+1}]$ for $j = 0, \dots, N$, with $p_3 = \mathcal{O}(\log \frac{1}{\epsilon} + \log \frac{1}{\delta})$. Then, for $t \in [\delta, T]$ and $\alpha \in (0, 2)$,

$$\left| \frac{1}{t^\alpha} - \left(\sum_{k=1}^{p_0} w_{l,k} e^{-s_{l,k} t} + \sum_{j=-M}^{-1} \sum_{k=1}^{p_1} s_{j,k}^{\alpha-1} w_{j,k} e^{-s_{j,k} t} + \sum_{j=0}^{N} \sum_{k=1}^{p_2} s_{j,k}^{\alpha-1} w_{j,k} e^{-s_{j,k} t} \right) \right| \le \epsilon.$$

2.2. Kernel-based transformation

Taking $\mathcal{I}_t^\alpha$ on both sides of (2.1), we can get

$$u(\mathbf{x}, t) + \int_0^t k(t-s) \mathcal{A} u(\mathbf{x}, s) ds = u(\mathbf{x}, 0) + \mathcal{I}_t^\alpha (f(\mathbf{x}, t)), \tag{2.3}$$

where

$$k(t-s) = \frac{(t-s)^{\alpha-1}}{\Gamma(\alpha)}, \quad \mathcal{I}_t^\alpha (f(\mathbf{x}, t)) = \frac{1}{\Gamma(\alpha)} \int_0^t (t-s)^{\alpha-1} f(\mathbf{x}, s) ds,$$

and we have used $\mathcal{I}_t^\alpha \mathcal{I}_t^\beta = \mathcal{I}_t^{\alpha+\beta}$, $\alpha, \beta > 0$.

We define the approximate solutions of (2.3) as $v(t)$, which satisfy

$$v(\mathbf{x}, t) + \int_0^t \hat{k}(t-s) \mathcal{A} v(\mathbf{x}, s) ds = g(\mathbf{x}, t), \ v(\mathbf{x}, 0) = u_0(\mathbf{x}), \tag{2.4}$$

where $g(\mathbf{x}, t) = u(\mathbf{x}, 0) + \mathcal{I}_t^\alpha (f(x, t))$, and $\hat{k}(t-s)$ is an approximation to $k(t-s)$.

The non-local problem (2.4) can be transformed into a system of integer order differential equations, which is shown in the following theorem.

Theorem 2.3. *Suppose $v(\mathbf{x}, t)$ is the solution of (2.4), and define*

$$\phi_i(\mathbf{x}, t) = \int_0^t e^{-s_i(t-s)} v(\mathbf{x}, s) ds. \tag{2.5}$$

$v(\mathbf{x}, t)$ is equivalent to the solutions $v(\mathbf{x}, t)$ of the following system

$$\begin{cases} v(\mathbf{x}, t) + \dfrac{1}{\Gamma(\alpha)} \displaystyle\sum_{i=1}^{N_p} w_i \mathcal{A}(\phi_i(\mathbf{x}, t)) = g(\mathbf{x}, t), \\ \dfrac{d\phi_i(\mathbf{x}, t)}{dt} + s_i \phi_i(\mathbf{x}, t) = v(\mathbf{x}, t), \quad i = 1, \dots, N_p, \end{cases} \tag{2.6}$$

with the initial conditions as $v(\mathbf{x}, 0) = u_0(\mathbf{x})$, $\phi_i(\mathbf{x}, t) = 0$, $i = 1, \dots, N_p$.

Proof. For simplicity, we denote $v(\mathbf{x}, t)$ by $v(t)$ here and below. The first equation of (2.6) can be derived by

$$\begin{aligned} g(t) &= v(t) + \frac{1}{\Gamma(\alpha)} \sum_{i=1}^{N_p} w_i \int_0^t e^{-s_i(t-s)} \mathcal{A} v(s) ds \\ &= v(t) + \frac{1}{\Gamma(\alpha)} \sum_{i=1}^{N_p} w_i \mathcal{A}(\phi_i(t)). \end{aligned}$$

For $i = 1, \dots, N_p$, we have

$$\frac{d\phi_i(t)}{dt} = e^{-s_i(t-s)} v(s)|_{s=t} - s_i \int_0^t e^{-s_i(t-s)} v(s) ds,$$

which implies

$$\frac{d\phi_i(t)}{dt} + s_i \phi_i(t) = v(t).$$

This completes the proof. □

Define the error of the approximate solution as $w(\mathbf{x}, t) = u(\mathbf{x}, t) - v(\mathbf{x}, t)$, where $u(\mathbf{x}, t)$ is the solutions of (2.3) and $v(\mathbf{x}, t)$ is the solutions of (2.6). We have the following error estimate.

Theorem 2.4. *Let $\mathcal{A}$ be a positive definite self-adjoint operator in a Hilbert space, ϵ is the desired precision. The kernel $k(t)$ is defined in (2.3). If the kernel approximation error satisfies the estimate*

$$|k(t) - \hat{k}(t)| \le \epsilon, \quad \forall t \in (0, T].$$

Then, it holds

$$\|w(\mathbf{x}, t)\|_{L^\infty} \le C_w \epsilon, \quad (x, t) \in \Omega \times [0, T],$$

where $C_w > 0$ is a constant independent of ϵ.

Proof. From (2.6), we can calculate

$$\begin{aligned} \phi_i(\mathbf{x}, t) &= e^{-\int_0^t s_i dt} \left[\int_0^t v(\mathbf{x}, s) e^{\int_0^s s_i d\xi} ds + C_0 \right] \\ &= e^{-s_i t} \left[\int_0^t v(\mathbf{x}, s) e^{s_i s} ds + C_0) \right], \end{aligned}$$

where $C_0 = 0$, which is calculated using the initial condition. Hence, we can get

$$v(\mathbf{x}, t) + \frac{1}{\Gamma(\alpha)} \sum_{i=1}^{N_p} \mathcal{A} \left(\int_0^t v(\mathbf{x}, s) w_i e^{-s_i(t-s)} ds \right) = g(\mathbf{x}, t),$$

which equals (2.4).

Subtracting equation (2.4) from (2.3), we get

$$w(\mathbf{x}, t) + \int_0^t k(t-s) \mathcal{A} w(\mathbf{x}, s) \, ds = F(\mathbf{x}, t),$$

where $w(\mathbf{x}, 0) = 0$ and

$$F(\mathbf{x}, t) = \int_0^t \left[k(t-s) - \hat{k}(t-s) \right] \mathcal{A} v(\mathbf{x}, s) \, ds.$$

Let $\psi(t-s) = k(t-s) - \hat{k}(t-s)$. Then, it holds that $|\psi(t-s)| < \epsilon$.

By equation (2.4) and regularity theory, we conclude that

$$\mathcal{A} v(\mathbf{x}, t) \in L^\infty(\Omega \times [0, T]).$$

Therefore, there exists a constant M such that

$$\|\mathcal{A} v(\mathbf{x}, t)\|_{L^\infty} \le M.$$

As a result, it holds that

$$\|F(\mathbf{x}, t)\|_{L^\infty} \le \int_0^t |\psi(t-s)| \cdot \|\mathcal{A} v(\mathbf{x}, t)\|_{L^\infty} ds < \epsilon M t \le C_w \epsilon,$$

where $C_w = MT$, $t \in [0, T]$.

Now, define

$$\mathcal{L} w(\mathbf{x}, t) = w(\mathbf{x}, t) + \int_0^t k(t-s) \mathcal{A} w(\mathbf{x}, t) \, ds,$$

where $\mathcal{L}$ is an operator . For constant $B = C_w \epsilon$, there is $\mathcal{L} B = B$ (since $\Delta B = 0$). By comparison principle, it holds that

$$\mathcal{L} w(\mathbf{x}, t) = F(\mathbf{x}, t) \le C_w \epsilon = \mathcal{L} B,$$

which implies

$$w(\mathbf{x}, t) \le C_w \epsilon, \quad \text{for } (x, t) \in (\Omega \times [0, T]).$$

Replacing $w(\mathbf{x}, t)$ with $-w(\mathbf{x}, t)$, we further have $-w(\mathbf{x}, t) \le C_w \epsilon$.

Therefore, we can get

$$\|w(\mathbf{x}, t)\|_{L^\infty} \le C_w \epsilon,$$

where C_w is independent of ϵ. Thus, we finish the proof. □

3. Kernel-based PINNs

We are now ready to present the framework of our kernel-based PINN method.

3.1. Kernel-based PINNs for linear time fractional differential equations

Here and below, let $D = \Omega \times (0, T]$ be a temporal-spatial domain, and $X = (\mathbf{x}, t)$ for $\mathbf{x} \in \Omega$ and $t \in (0, T]$ be the set of sample points.

The system (2.6) can be expressed as the following PDEs:

$$\begin{cases} \mathcal{F}(X; v(X); \phi(X)) = 0, & X \in D, \\ \mathcal{B}(X; v(X); \phi(X)) = 0, & X \in \partial D, \end{cases} \tag{3.1}$$

where $\partial D = \partial\Omega \cup (\{t = 0\} \times \Omega)$, $\mathcal{F}$ represents a linear or nonlinear operator, $\mathcal{B}$ represents the boundary operator, $v(X) = v(\mathbf{x}, t)$ and $\phi(X) = [\phi_i(\mathbf{x}, t)]_{i=1}^{N_p}$ are the numerical solutions to be solved.

Following the idea of PINNs, the solutions of (3.1) are approximated by a deep neural network with outputs $v(X; \theta)$ and $\phi(X; \theta)$, where θ are

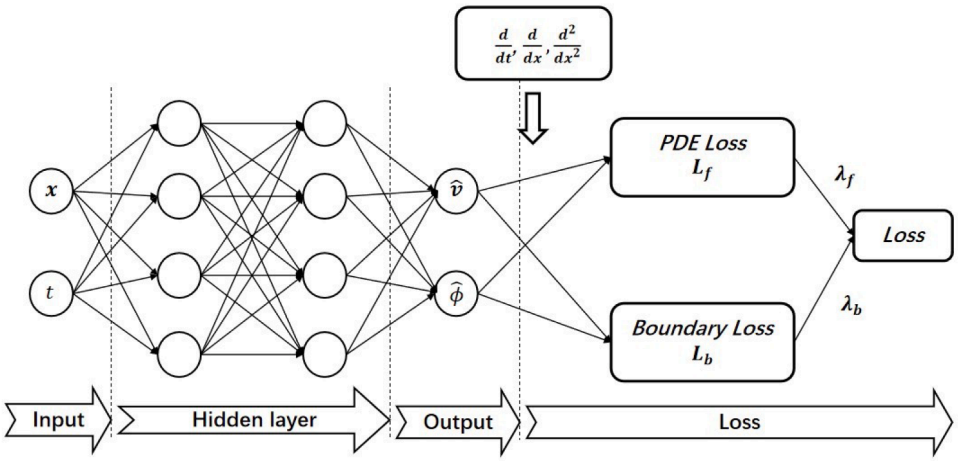


Fig. 1. Network architecture of the proposed kPINN.

the parameters of the neural network. Since we can not get all the points in $\mathcal{D}$, we restrict the solutions of the neural network as $\hat{v}(X;\theta)$, $\hat{\phi}(X;\theta)$ at some collocation points. The points in the domain $\mathcal{D}$, on the boundary $\partial\mathcal{D}$ are $P_f = \{X_1, X_2, \dots, X_{|P_f|}\}$ and $P_b = \{X_1, X_2, \dots, X_{|P_b|}\}$, respectively. We consider the loss function as

$$\mathcal{L}(\theta) = \lambda_f \mathcal{L}_f(P_f;\theta) + \lambda_b \mathcal{L}_b(P_b;\theta),$$

where

$$\mathcal{L}_f(P_f;\theta) = \frac{1}{|P_f|}\sum_{X\in P_f} \|\mathcal{F}(X;\hat{v}(X;\theta);\hat{\phi}(X;\theta))\|_2^2,$$

$$\mathcal{L}_b(P_b;\theta) = \frac{1}{|P_b|}\sum_{X\in P_b} \|\mathcal{B}(X;\hat{v}(X;\theta);\hat{\phi}(X;\theta))\|_2^2,$$

with $\|\cdot\|_2$ being L_2 norm and λ_f and λ_b being super-parameters for balancing. The network parameters θ are determined by calculating the best parameters of the soft constrained optimization problem

$$\min_{\theta\in\Theta} \mathcal{L}(\theta) = \min_{\theta\in\Theta} \big(\lambda_f \mathcal{L}_f(P_f;\theta) + \lambda_b \mathcal{L}_b(P_b;\theta)\big),$$

where Θ is the parameter space. The kPINNs framework is presented in Fig. 1.

For our problem (2.6), the loss function $\mathcal{L}_f$ is divided into several parts. We define

$$\mathcal{L}_{\hat{v}}(P_f;\theta) = \sum_{X\in P_f} \left\| \hat{v}(X;\theta) + \frac{1}{\Gamma(\alpha)}\sum_{i=1}^{N_\mathcal{P}} \omega_i \mathcal{A}(\hat{\phi}_i(X;\theta)) - g(X) \right\|_2^2,$$

$$\mathcal{L}_{\hat{\phi}_i}(P_f;\theta) = \sum_{X\in P_f} \left\| \frac{d\hat{\phi}_i(X;\theta)}{dt} + s_i\hat{\phi}_i(X;\theta) - \hat{v}(X;\theta) \right\|_2^2, \quad i = 1, \dots, N_\mathcal{P}.$$

Then, the loss function is calculated as

$$\mathcal{L}_f(P_f;\theta) = \mathcal{L}_{\hat{v}}(P_f;\theta) + \sum_{i=1}^{N_\mathcal{P}} \lambda_{\phi_i}\mathcal{L}_{\hat{\phi}_i}(P_f;\theta),\ \lambda_{\phi_i} = \frac{w_i}{W},\ W = \sum_{i=1}^{N_\mathcal{P}} w_i, \quad (3.2)$$

where the weights and nodes of the SoEs approximation are calculated in Lemma 2.2.

Remark 3.1. Local systems (2.6) consist of a family of integer order stiff differential equations. This is because the coefficients s_i and w_i vary widely. To overcome the difficulties from the stiffness, we introduce the weights $\frac{w_i}{W}$ into the super-parameters for balancing the loss function. The weights w_i are accompanied with the SoEs approximation. By introducing the weights, we normalize each function in our system and accelerate the convergence of PINNs. Due to the crucial weights, the kPINNs can work well for systems (2.6).

The error estimation of neural networks approximations are shown in the following lemma [65].

Lemma 3.1. *Let $\Omega \subseteq \mathbb{R}^d$, $d \in \mathbb{N}$, be a bounded Lipschitz domain. Moreover, let $k, m \in \mathbb{N}_0$ with $k > m$. Then, for every $n \in \mathbb{N}$ and every $u \in H^k(\Omega)$, there exists a fully-connected tanh-network $u(\theta_n) \in H^m(\Omega)$ with parameter space Θ_n of dimension $\mathcal{O}(n)$ such that it holds*

$$\|u - u(\theta_n)\|_{H^m(\Omega)} \lesssim \left(\frac{1}{n}\right)^{\frac{k-m-\mu}{d}} \|u\|_{H^k(\Omega)},$$

where $\mu > 0$ can be arbitrarily small.

Thus, for a fully-connected neural network with parameter space Θ_n of dimension $\mathcal{O}(n)$, the error estimation of the kPINNs is given as the following theorem.

Theorem 3.2. *Let the assumption in Lemma 3.1 holds, $u(\mathbf{x},t)$, $v(\mathbf{x},t)$ be the solutions of (2.1) and (2.6). $\hat{v}(\mathbf{x},t;\theta)$ denote the kPINNs approximation with parameter $\theta \in \Theta_n$. Then the following error estimation holds,*

$$\|u(\mathbf{x},t) - \hat{v}(\mathbf{x},t;\theta)\|_2 \le C_1\epsilon + C_2 n^{-\frac{1-\mu}{d}} \|v\|_{H^1(\Omega)},$$

where μ can be arbitrarily small, $v = v(\mathbf{x},t)$, and C_1, C_2 are constants independent of the neural parameters θ.

Proof. The error equation can be written as

$$\begin{aligned}\|u(\mathbf{x},t) - \hat{v}(\mathbf{x},t;\theta)\|_2 &\le \|u(\mathbf{x},t) - v(\mathbf{x},t)\|_2 + \|v(\mathbf{x},t) - \hat{v}(\mathbf{x},t;\theta)\|_2 \\ &\le C_1\|u(\mathbf{x},t) - v(\mathbf{x},t)\|_{L^\infty} + \|v(\mathbf{x},t) - \hat{v}(\mathbf{x},t;\theta)\|_2,\end{aligned}$$

where we have used Theorem 2.4 and C_1 is a constant independent of the neural parameters θ. By using Lemma 3.1, we can get the result. □

Next, we are ready to present the extension and implementation of the proposed kPINNs to general time fractional differential equations and Volterra integral-differential equations.

3.2. Kernel-based PINNs for general time fractional differential equations

We now consider the general time fractional differential Eqs. (1.1) with $\alpha \in (n, n+1)$, $n \in \mathbb{N}$. Taking $\mathcal{I}_t^{\alpha-n}$ on both sides of (1.1), we can get

$$\frac{d^n u(\mathbf{x},t)}{dt^n} = \frac{d^n u(\mathbf{x},0)}{dt^n} + \int_0^t \hat{k}(t-s)\big(f(u(\mathbf{x},t),\mathbf{x},t) - \mathcal{A}v(\mathbf{x},s)\big)ds. \quad (3.3)$$

The corresponding approximate solutions $v(\mathbf{x},t)$ satisfy

$$\frac{d^n v(\mathbf{x},t)}{dt^n} = \frac{d^n u(\mathbf{x},0)}{dt^n} + \int_0^t \hat{k}(t-s)\big(f(v(\mathbf{x},s),\mathbf{x},s) - \mathcal{A}v(\mathbf{x},s)\big)ds. \quad (3.4)$$

Let

$$\phi_i(\mathbf{x},t) = \int_0^t e^{-s_i(t-s)}\big(f(v(\mathbf{x},s),\mathbf{x},s) - \mathcal{A}v(\mathbf{x},s)\big)ds.$$

The nonlocal problem (3.3) can be rewritten as the local systems

$$\begin{cases} \dfrac{d^n v(\mathbf{x},t)}{dt^n} - \sum\limits_{i=1}^{N_\mathcal{P}} w_i \phi_i(\mathbf{x},t) = u_1(\mathbf{x},t), \\ \dfrac{d\phi_i(\mathbf{x},t)}{dt} + s_i \phi_i(\mathbf{x},t) = f(v(\mathbf{x},t),\mathbf{x},t) - \mathcal{A}v(\mathbf{x},t), \quad i = 1,\dots,N_\mathcal{P}, \end{cases} \tag{3.5}$$

where $u_1(\mathbf{x},t) = \frac{d^n u(\mathbf{x},0)}{dt^n}$ is a known function.

Using the network architecture in Fig. 1, we can calculate the solutions of (3.5). Let the solutions of our neural network be $\hat{v}(X;\theta)$ and $\hat{\phi}_i(X;\theta)$, $i = 1,\dots,N_\mathcal{P}$, let the sets of collocation points in the domain, boundary, and initial layers be P_f, P_b, P_{in} and the numbers of these sets be $|P_f|$, $|P_b|$, $|P_{in}|$. We define each term of the loss function as

$$\mathcal{L}_{\hat{v}}(P_f;\theta) = \frac{1}{|P_f|} \sum_{X\in P_f} \left\| \frac{d^n \hat{v}(X;\theta)}{dt^n} - \sum_{i=1}^{N_\mathcal{P}} w_i(\hat{\phi}_i(X;\theta)) - u_1(X;\theta) \right\|_2^2, \tag{3.6}$$

$$\mathcal{L}_{\hat{\phi}_i}(P_f;\theta) = \frac{1}{|P_f|} \sum_{X\in P_f} \left\| \frac{d\hat{\phi}_i(X;\theta)}{dt} + s_i\hat{\phi}_i(X;\theta) - f(\hat{v}(X;\theta),X) + \mathcal{A}\hat{v}(X;\theta) \right\|_2^2, \tag{3.7}$$

$$\mathcal{L}_b(P_f;\theta) = \frac{1}{|P_b|} \sum_{X\in P_b} \|\hat{v}(X;\theta) - 0\|_2^2, \tag{3.8}$$

$$\mathcal{L}_{in}(P_{in};\theta) = \frac{1}{|P_{in}|} \sum_{X\in P_{in}} \left(\|\hat{v}(X;\theta) - u_0(X;\theta)\|_2^2 + \sum_{i=1}^{N_\mathcal{P}} \|\hat{\phi}_i(X;\theta) - 0\|_2^2 \right). \tag{3.9}$$

The network updates the parameters θ by computing the loss function.

$$\mathcal{L} = \lambda_v \mathcal{L}_{\hat{v}}(P_f;\theta) + \sum_{i=1}^{N_\mathcal{P}} \lambda_i \mathcal{L}_{\hat{\phi}_i}(P_f;\theta) + \lambda_b \mathcal{L}_b(P_f;\theta) + \lambda_{in}\mathcal{L}_{in}(P_f;\theta), \tag{3.10}$$

where λ_v, λ_b, λ_{in}, λ_i, $i = 1,\dots,N_\mathcal{P}$ are weights.

3.3. Kernel-based PINNs for Volterra integral-differential equations

We consider some typical Volterra integral equations as follows

$$\frac{d^n u(\mathbf{x},t)}{dt^n} + \int_0^t k(t-s)\mathcal{A}u(\mathbf{x},t)ds = f(u(\mathbf{x},t),\mathbf{x},t), \quad n \geq 0,\ n \in \mathbb{Z}. \tag{3.11}$$

Using the SoEs method to approximate the kernel $k(t-s)$, the approximate solutions $v(\mathbf{x},t)$ satisfy

$$\frac{d^n v(\mathbf{x},t)}{dt^n} + \int_0^t \hat{k}(t-s)\mathcal{A}v(\mathbf{x},t)ds = f(v(\mathbf{x},t),\mathbf{x},t). \tag{3.12}$$

From Theorem 2.3, the non-local problem (3.11) can be approximated by the following local system.

$$\begin{cases} \dfrac{d^n v(\mathbf{x},t)}{dt^n} + \sum\limits_{i=1}^{N_\mathcal{P}} w_i \mathcal{A}(\phi_i(\mathbf{x},t)) = f(v(\mathbf{x},t),\mathbf{x},t), \\ \dfrac{d\phi_i(\mathbf{x},t)}{dt} + s_i\phi_i(\mathbf{x},t) = v(\mathbf{x},t), \quad i = 1,\dots,N_\mathcal{P}, \end{cases} \tag{3.13}$$

where $\phi_i(\mathbf{x},t)$ is defined in (2.5).

Similarly, we solve (3.13) using the neural network. We define each term of the loss function as (3.8), (3.9) and

$$\mathcal{L}_{\hat{v}}(P_f;\theta) = \frac{1}{|P_f|} \sum_{X\in P_f} \left\| \frac{d^n \hat{v}(X;\theta)}{dt^n} + \sum_{i=1}^{N_\mathcal{P}} w_i \mathcal{A}(\hat{\phi}_i(X;\theta)) - f(\hat{v}(X;\theta),X) \right\|_2^2, \tag{3.14}$$

$$\mathcal{L}_{\hat{\phi}_i}(P_f;\theta) = \frac{1}{|P_f|} \sum_{X\in P_f} \left\| \frac{d\hat{\phi}_i(X;\theta)}{dt} + s_i\hat{\phi}_i(X;\theta) - \hat{v}(X;\theta) \right\|_2^2. \tag{3.15}$$

Then, we update the parameters θ by again minimizing the loss function (3.10).

Table 1
The strategy of choosing learning rates.

Learning rate (η)	steps
10^{-2}	$0 \leq step \leq 10000$
10^{-3}	$10000 < step \leq 40000$
10^{-4}	$40000 < step \leq 90000$

4. Numerical experiments

In this section, we solve several different problems using kPINNs. Moreover, we compare our methods with fPINNs [57] using the predicted error $\mathcal{L}_{pred}$ and L_2 relative error ε,

$$\mathcal{L}_{pred} = \frac{1}{|P|} \sum_{X_i\in P} \|u(X_i) - \hat{v}(X_i;\theta)\|_2^2,$$

$$\varepsilon = \frac{\sqrt{\sum_{X_i\in P} \|u(X_i) - \hat{v}(X_i;\theta)\|_2^2}}{\sqrt{\sum_{X_i\in P} \|u(X_i)\|_2^2}},$$

where P represents the set of points for predicting our methods. The w_i and s_i for different experiments are all listed in Appendix A. All time contractions measure the GPU times. The numerical experiments were performed on a laptop equipped with an NVIDIA GeForce RTX 3060 processor. The codes run on PyTorch version 2.4.1 and Python version 3.12.3. We present a detailed algorithm pseudocode in Appendix B.

Example 4.1. We test the following time fractional differential equation

$$\partial_t^\alpha u(\mathbf{x},t) - \nabla\cdot(a(\mathbf{x})\nabla u(\mathbf{x},t)) + c(\mathbf{x})u(\mathbf{x},t) = f(\mathbf{x},t) \tag{4.1}$$

with the exact solution

$$u(\mathbf{x},t) = (t^\alpha + t^2 + t^{3+\alpha})\left(\prod_{i=1}^d \sin(\pi x_i)\right), \quad t \in (0,T],\ x_i \in [-1,1],$$

for $d = 1,2,3,4$, where $T = 1$, $\alpha \in (0,1)$, $a(\mathbf{x}) = \prod_{i=1}^d \sin(\pi x_i)$ and $c(x) = 4$. The boundary conditions, the initial condition, and the source term are given by the exact solution.

Accuracy tests. For each training step, we randomly select the training points from the entire space. In the one-dimensional problem, we let $|P_f| = 1000$, $|P_b| = 200$ and $|P_{in}| = 500$. The layer of our neural network is $[2] + [32] * 3 + [1 + \mathcal{N}_\mathcal{P}]$. In the two-dimensional problem, we let $|P_f| = 1500$, $|P_b| = 800$ and $|P_{in}| = 500$ in each training step, and the hidden layers as $[32] * 3$. In the three-dimensional problem, we let $|P_f| = 3000$, $|P_b| = 2000$ and $|P_{in}| = 1000$ in each training step, and the hidden layers as $[32] * 4$. The learning rates are changing with different steps, and the strategy is presented in Table 1. For the four-dimensional problem, we set different neural networks for different α. For $\alpha = 0.3$, we let $|P_f| = 2000$, $|P_b| = 2000$, $|P_{in}| = 2000$ and the hidden layers as $[32] * 4$. For $\alpha = 0.5,\ 0.8$, we let $|P_f| = 1500$, $|P_b| = 1000$, $|P_{in}| = 1000$ and the hidden layers as $[32] * 5$. All problems are solved following the same learning rate strategy. Furthermore, we compute the L_2 relative error by selecting 500, 1000, 3000, 5000 nodes for 1,2,3,4-dimensional problems in a global equidistant range and present the results in Table 2. Clearly, the predicted errors can reach $\mathcal{O}(10^{-3})$ and the L_2 relative errors can reach $\mathcal{O}(10^{-2})$. The results show the effectiveness of our method.

Comparison of the Computational time. We compare the computational time of kPINNs and fPINNs. For the fPINNs methods, the time fractional derivative is discretized by using the uniform L1 scheme [66]. We calculate the L_2 relative error at $T = 1$, and divide the time interval $[0,1]$ into $\tau = T/M$, $t_k = \tau k$, $k = 0,1,2,\dots,M$. The numerical scheme for the fPINNs is

$$\partial_t^\alpha u(\mathbf{x}_i,t_M;\theta) \approx \sum_{k=1}^M a_{M-k}^{(M)}(\hat{v}(\mathbf{x}_i,t_k;\theta) - \hat{v}(\mathbf{x}_i,t_{k-1};\theta)), \tag{4.2}$$

Table 2
The predicted loss and L_2 relative error of 1,2,3,4-dimensional problems for Example 4.1.

	1D		2D		3D		4D	
α	$\mathcal{L}_{pred}$	ϵ	$\mathcal{L}_{pred}$	ϵ	$\mathcal{L}_{pred}$	ϵ	$\mathcal{L}_{pred}$	ϵ
0.3	5.54e-03	7.22e-02	2.04e-03	5.12e-02	3.26e-03	7.11e-02	2.98e-03	7.29e-02
0.5	1.16e-03	3.49e-02	8.62e-02	5.15e-02	7.36e-04	3.59e-02	4.10e-03	9.09e-02
0.8	2.39e-04	1.70e-02	4.92e-04	2.87e-02	5.60e-04	3.38e-02	1.98e-04	2.15e-02

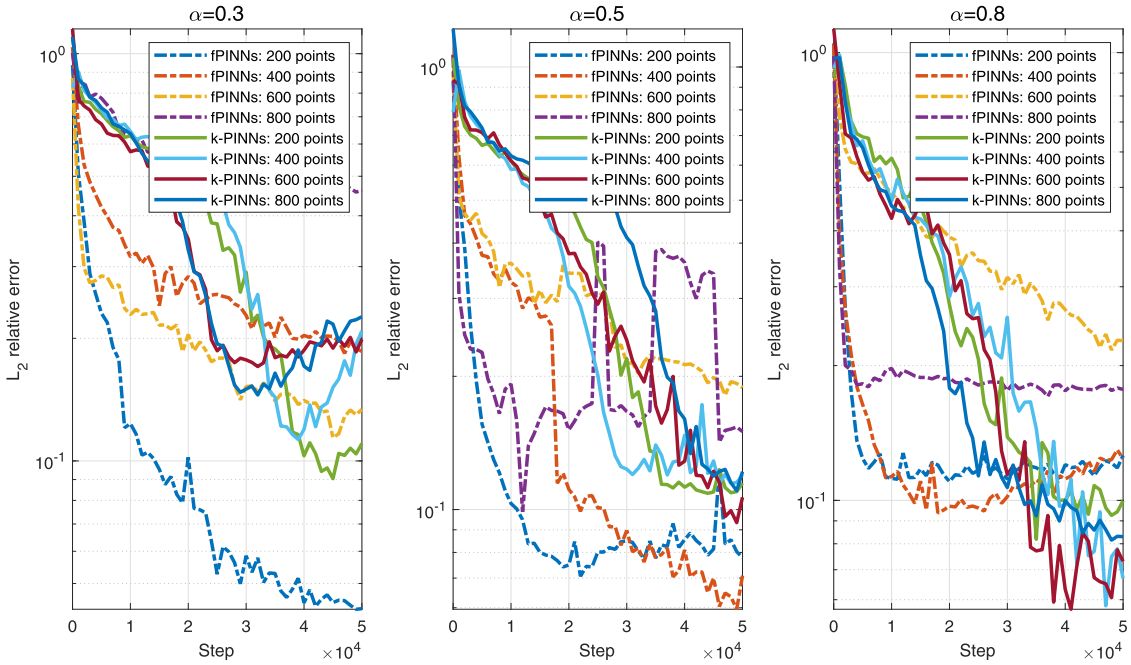


Fig. 2. L_2 relative errors of the fPINN and kPINNs for a two-dimensional problem with $\alpha = 0.3$(left), $\alpha = 0.5$(middle), $\alpha = 0.8$(right) with $\eta = 1 \times 10^{-3}$, training steps = 5×10^4, Adam optimizer, batch size = $M = 200, 400, 600, 800$ for Example 4.1.

where $\mathbf{x}_i$ represents the ith sample point in space, and

$$a_{M-k}^{(M)} = \frac{1}{\tau^\alpha \Gamma(2-\alpha)}\left[(M-k+1)^{1-\alpha} - (M-k)^{1-\alpha}\right].$$

The hidden layers of the neural networks are defined as $[32] \times 3$ for one and two dimensions, and $[32] \times 4$ for three dimensions, respectively. The learning rate and steps are chosen as 10^{-3} and 5×10^4. The time interval is divided into $200, 400, 600, 800$ for the fPINNs and M points in the domain Ω and the initial condition for our method. The number of points on the boundary $\partial\Omega$ is chosen to be 500. The L_2 relative errors are calculated with 100 sample points $\mathbf{x}_i$ at $T = 1$, and the results with different α are presented in Fig. 2 for a three-dimensional problem. The results show that when $\alpha \geq 0.5$, the fPINNs method is easier to overfit. The L_2 relative errors can only reach 10^{-1}. While the errors of kPINNs can be 5×10^{-2} in a high-dimensional calculation.

We also present the computational time in Fig. 3. The results show that the computational time of kPINNs increases much more slowly while increasing the number of training points in each training step. The computational time of kPINNs remains below 10^4 seconds even for a three-dimensional problem, while the computational time of fPINNs can exceed 10^5 s. These results indicate that the kPINNs are more competitive and save a considerable amount of computational time.

Example 4.2. We test the following time fractional differential Eq. (4.1) with $\alpha \in (1, 2)$,

$$a(\mathbf{x}) = e^{\sum_{i=1}^d -\frac{1}{2}x_i^2}, \quad c(\mathbf{x}) = a(\mathbf{x}) \sum_{i=1}^d (1 - x_i^2),$$

The boundary conditions and the initial conditions are given by the exact solution

$$u(\mathbf{x}, t) = (1 + t^{\alpha+1} + t^{2+\alpha})(\prod_{i=1}^d (1 - x_i^2)), \quad t \in (0, 1], \ x_i \in [-1, 1],$$

for $d = 1, 2, 3, 5$. The source term is calculated according to the exact solution.

Accuracy tests. The training points are randomly selected from the entire space in each training step. For the one-, two-, three- and five dimensional problems, the numbers of collocation points for each step are set as {500, 200, 200}, {1500, 800, 500}, {1000, 600, 500}, {3000, 2000, 1000} for the domain, boundary and initial layer, respectively. The hidden layers of the network are defined as $[32] * 3$ for one- and two-dimensional problems, $[32] * 4$ for three-dimensional problems and $[32] * 5$ for five-dimensional problems. The learning rates are selected and shown in Table 1. The L_2 relative error and the corresponding predicted loss are calculated by choosing 5000, 5000, 10000, 100,000 nodes in a global equidistant range. The numerical results are presented in Table 3. For 1-, 2- 3- and 5-dimensional problems, it can be observed that the predicted errors are lower than 5×10^{-3} and the L_2 relative errors are lower than 1×10^{-1}. These results show that the kPINNs can be relatively accurate. Additionally, the errors of our method under different depths and widths of neural networks are presented in Table 4. It implies that the kPINNs are effective for solving the problem, when we choose some appropriately broad parameters.

Comparison of Computational time. We compare the computational time of kPINNs and fPINNs and show the computational time when $T = 1$. We define $\tau = T/M$, $t_k = k\tau$, $k = 0, 1, \ldots, M$ and $\delta_t \hat{v}(t_{k-1/2}) = (\hat{v}(t_k) - \hat{v}(t_{k-1}))/\tau$. For the fPINN methods at each sample

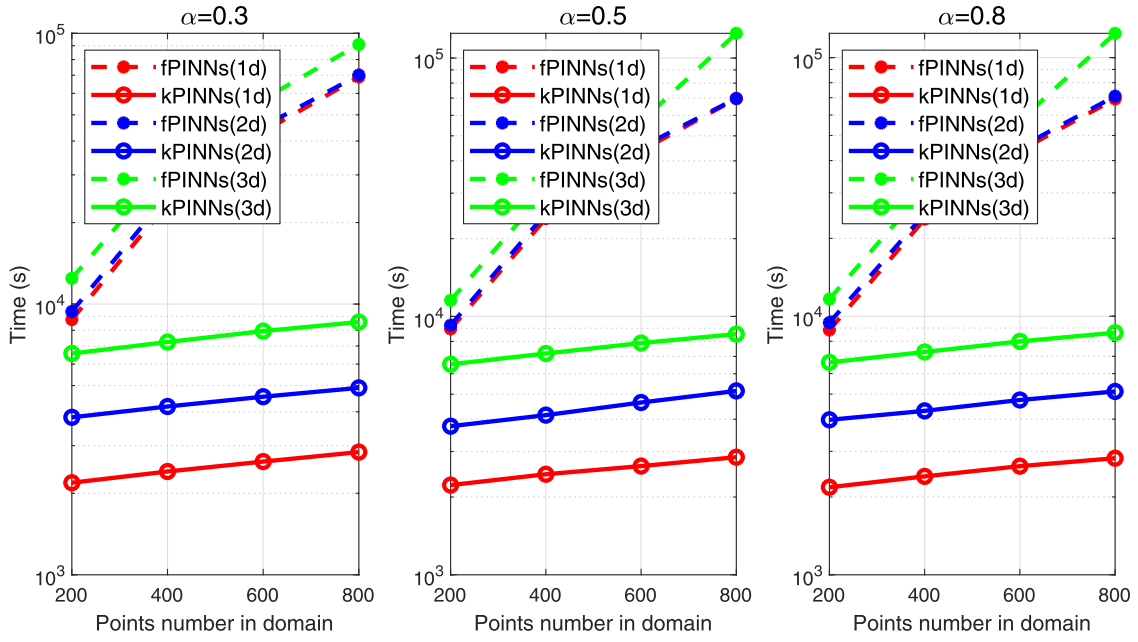


Fig. 3. Computational time of the fPINNs and kPINNs with $\alpha = 0.3$(left), $\alpha = 0.5$(middle), $\alpha = 0.8$(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600, 800$ for Example 4.1.

Table 3
The predicted loss and L_2 relative error of 1,2,3,5-dimensional problems for Example 4.2.

	1D		2D		3D		5D	
α	$\mathcal{L}_{pred}$	ε	$\mathcal{L}_{pred}$	ε	$\mathcal{L}_{pred}$	ε	$\mathcal{L}_{pred}$	ε
1.2	6.14e-04	2.32e-02	8.36e-04	3.21e-02	1.33e-03	4.50e-02	9.05e-04	4.23e-02
1.5	4.46e-04	2.65e-02	2.00e-03	5.18e-02	1.52e-03	4.47e-02	2.07e-03	6.70e-02
1.7	8.36e-04	2.87e-02	2.61e-04	1.92e-02	1.90e-03	5.76e-02	2.13e-03	6.97e-02

Table 4
The predicted loss and L_2 relative error of 2-dimensional problems with $\alpha = 1.5$, different widths and depth of neural networks for Example 4.2.

width	[32]			[32]	[64]	[128]
depth	3	5	7	3		
$\mathcal{L}_{pred}$	2.00e-3	3.17e-4	8.34e-4	2.00e-3	8.86e-4	3.01e-3
ε	5.18e-2	2.06e-2	3.34e-2	5.18e-2	3.45e-2	6.35e-2

point $\mathbf{x}_i$ in space, the time fractional derivative is discretised by the uniform L1 scheme,

$$\partial_t^\alpha \hat{v}(x_i,t;\theta) \approx b_0^{(M)}\delta_t\hat{v}(\mathbf{x}_i,t_{M-1/2};\theta) - \sum_{k=1}^{M-1}(b_{M-k-1}^{(M)} - b_{M-k}^{(M)})\delta_t\hat{v}(\mathbf{x}_i,t_{k-1/2};\theta) - b_{M-1}^{(M)}\frac{\partial\hat{v}(\mathbf{x}_i,0;\theta)}{\partial t},$$

where $b_k^{(M)} = \frac{\tau^{1-\alpha}}{\Gamma(3-\alpha)}[(k+1)^{2-\alpha} - k^{2-\alpha}]$.

Let the learning rate be 1×10^{-3}, steps be 50000. We define 3 hidden layers with 32 neurons each for 1,2-dimensional problems and 4 hidden layers with 32 neurons each for 3-dimensional problems. With different α and different time points $M = 200 \times k,\ k = 1, 2, 3, 4$, we compare the L_2 relative errors in 100 sample points. The sample points for training our network on the boundary and initial layer are 500 and M, respectively. We present the contrasts of the computational time of different dimension in Fig. 4 and the evolution of L_2 relative errors with setting step as the x-coordinate in Figs. 5 and 6 for two and three dimensional problem. The figures show that the L_2 relative errors of kPINNs can only be $\mathcal{O}(10^{-2})$, while fPINNs can be $\mathcal{O}(10^{-1})$. Moreover, as the number of nodes increases, the computational time of kPINNs changes more gradually compared to fPINNs. Furthermore, we present the evolution of L_2 relative errors with setting time(s) as the x-axis for two-dimensional problem in Fig. 7. It shows that kPINNs can achieve higher accuracy in shorter time, compared to fPINNs. These results imply that our kPINNs offer substantial improvements in both computational efficiency and accuracy.

Example 4.3. (Volterra integral equations) Consider the following quasi-static visco-elasticity problems [1]

$$-\nabla^2 u(t,\mathbf{x}) = f(t,\mathbf{x}) - \int_0^t k(t-s)\nabla^2 u(s,\mathbf{x})ds, \quad t \in (0,T],\ \mathbf{x} \in \Omega^d, \tag{4.3}$$

for $d = 1, 3, 4$. The boundary conditions and the initial conditions are given by the exact solution

$$u(t,\mathbf{x}) = (t^{1-\alpha} + t^3) \cdot \Pi_{k=1}^d \cosh(x_k).$$

Accuracy tests. The training sample points are chosen randomly from the entire space. For the one-dimensional case, we let $|P_f|,\ |P_b|,\ |P_{in}| = 400,\ 50,\ 100$, and for the three- and four-dimensional case, we let $|P_f|,\ |P_b|,\ |P_{in}| = 800,\ 200,\ 400$. The hidden layers for several cases are $[20] * 3$, and the strategy of choosing the learning rates matches Table 1. The predicted errors and L_2 relative errors with $T = 1$ are listed in Table 5. The evolution of L_2 relative error from multiple iterations is shown in Fig. 8(a) by choosing learning rate as 0.001 and training 50,000 epoches. Such figure validate the reproducibility of kPINNs. The evolutions of L_2 relative error during our training for solving four-dimensional problems with different α are presented in Fig. 8(b). We can observe that our method converges quickly and the L_2 relative errors of numerical solutions $\hat{v}$ can reach notably small values, which are $\mathcal{O}(10^{-2})$.

Comparison of Computational time. We define steps $= 30000$, lr $= 1 \times 10^{-3}$, hidden layer $= [32] \times 3$ to compute the computational costs

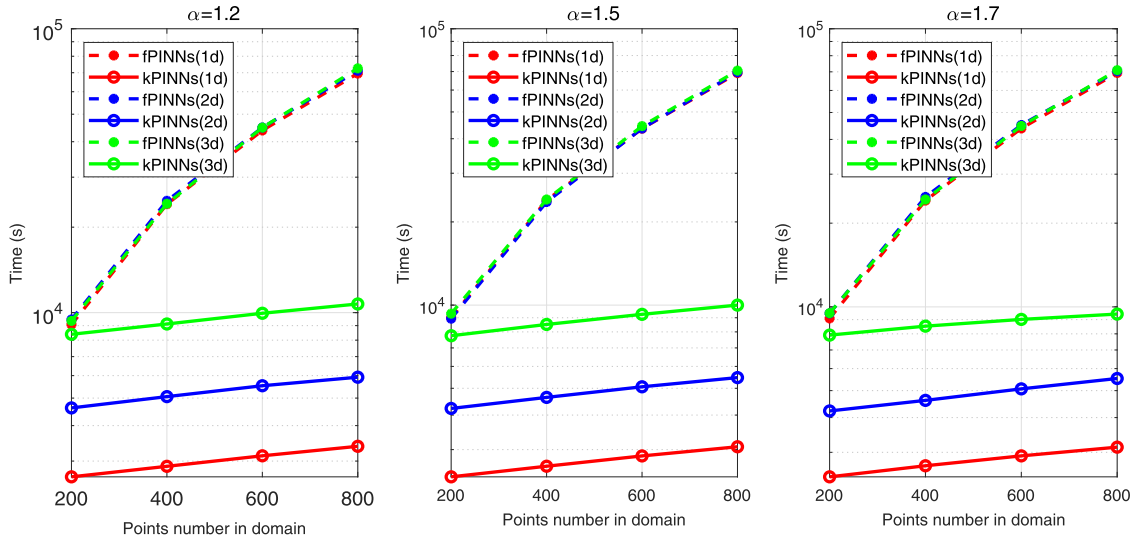


Fig. 4. Computational time of the fPINNs and kPINNs with $\alpha = 1.2$(left), $\alpha = 1.5$(middle), $\alpha = 1.7$(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600, 800$ for Example 4.2.

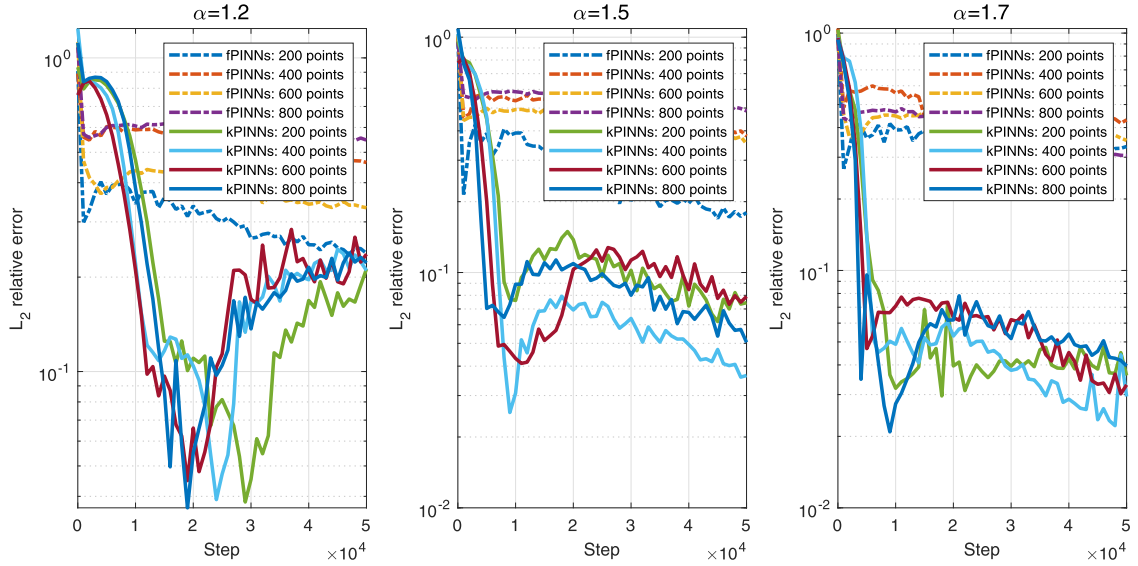


Fig. 5. L_2 relative errors of the fPINNs and kPINNs for two-dimensional problem with $\alpha = 1.2$(left), $\alpha = 1.5$(middle), $\alpha = 1.7$(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600, 800$ for Example 4.2.

Table 5
The predicted loss and L_2 relative error of 1,3,4-dimensional problems for Example 4.3.

	1D		3D		4D	
α	$\mathcal{L}_{pred}$	ε	$\mathcal{L}_{pred}$	ε	$\mathcal{L}_{pred}$	ε
0.3	3.49e-03	1.98e-02	1.68e-04	5.97e-03	1.56e-04	1.00e-02
0.5	6.83e-03	2.72e-02	1.16e-03	1.53e-02	5.82e-03	5.89e-02
0.7	1.51e-02	3.92e-02	3.19e-03	2.45e-02	1.29e-03	2.63e-02

and accuracy of our method and fPINNs. For the fPINNs method, we define $T = 1$, $\tau = T/M$, $t_k = k\tau$, $k = 0, 1, \ldots, M$. Then, the numerical solutions at each sample point x_i of (4.3) can be calculated by

$$-\nabla^2 u(x_i, t; \theta) = f(x_i, t; \theta) - \sum_{k=1}^{n} \left[\sum_{j=1}^{G_p} (t - s_{k,j})^{-\alpha} \cdot \nabla^2 u(x_i, s_{k,j}; \theta) \right],$$

where G_p are the Gauss point numbers, $s_{k,j}$ is the $j-$th Gauss point in the interval $[t_{k-1}, t_k]$. We define $M = 200, 400, 600$, $G_p = 4$, and compare the L_2 relative errors in 3000 sample points at $t = 1$ for the three-dimensional problem. We set M points in the domain, 500 points on the boundary and M points in the initial time layer to do each training of the kPINNs. We present the evolution of L_2 relative error in Fig. 9 and the comparison of the computational time in Fig. 10. We can observe that fPINNs exhibit overfitting at fewer steps, whereas the errors do not decay. In contrast, the L_2 relative errors of kPINNs achieve an improvement and demonstrate a decreasing trend. Furthermore, the computational time of fPINNs exceeds 10^5 s when selecting 600 points in each training. In contrast, the computational time of kPINNs remain below 10^4 s. These results demonstrate that our kPINNs are more competitive.

Example 4.4. Consider the two-dimensional time fractional Allen-Cahn equation

$$\partial_t^\alpha u = \varepsilon^2 \Delta u + u - u^3 + g, \quad \text{on } [-\pi, \pi]^2 \times (0, T],$$

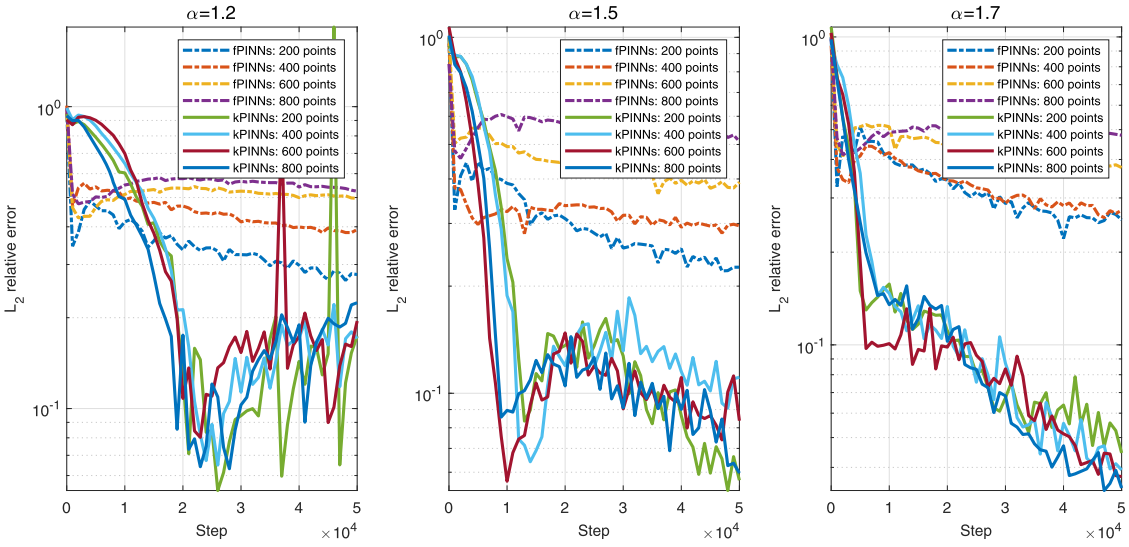


Fig. 6. L_2 relative errors (x-axis: steps) of the fPINNs and kPINNs for three-dimensional problem with $\alpha = 1.2$(left), $\alpha = 1.5$(middle), $\alpha = 1.7$(right), $\eta = 1 \times 10^{-3}$, training steps = 5×10^4, Adam optimizer, batch size = $M = 200, 400, 600, 800$ for Example 4.2.

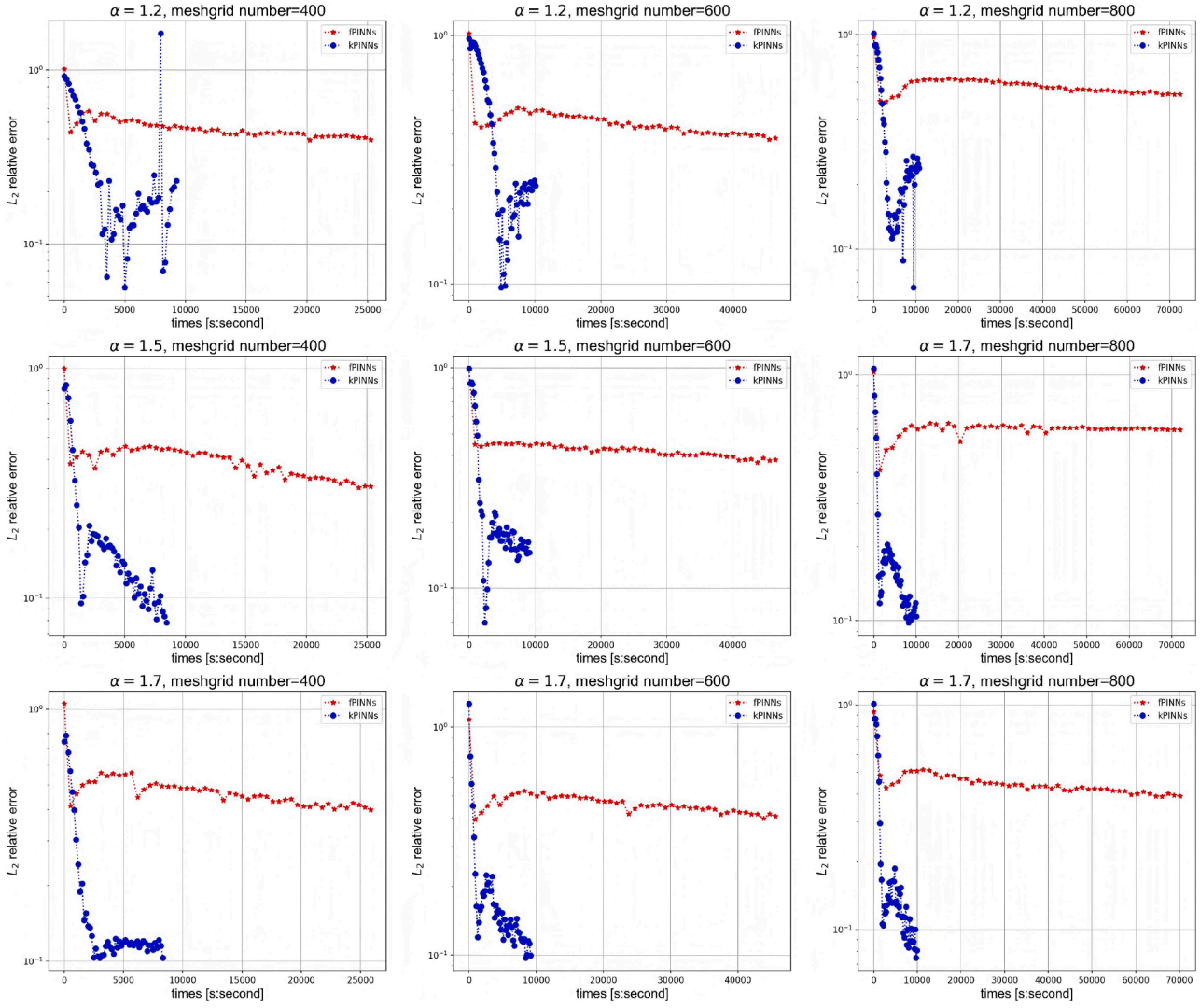
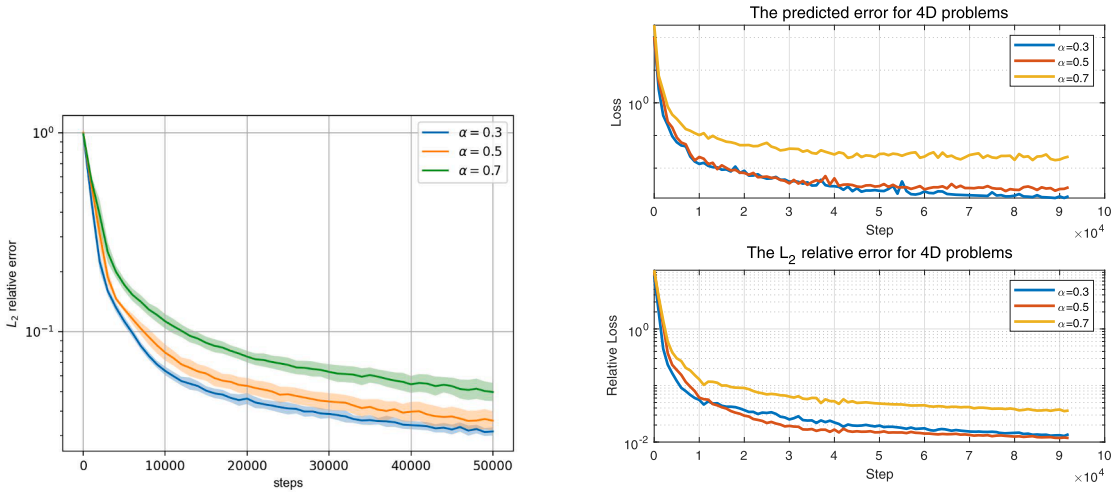


Fig. 7. L_2 relative errors (x-axis: time) of the fPINNs and kPINNs for two-dimensional problem with $\alpha = 1.2,\ 1.5,\ 1.7$, $\eta = 1 \times 10^{-3}$, training steps = 5×10^4, Adam optimizer, batch size = $M = 400, 600, 800$ for Example 4.2.



(a) (b)

Fig. 8. Solving four-dimensional problem with $\alpha = 0.3,\ 0.5,\ 0.7$, $|P_f|,\ |P_b|,\ |P_{in}| = 400,\ 50,\ 100$ (a) multiple runs (9 times) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, (b) evolutions of predicted loss (up) and L_2 relative error(down) with Adam optimizer, η and training steps following Table 1 for Example 4.3.

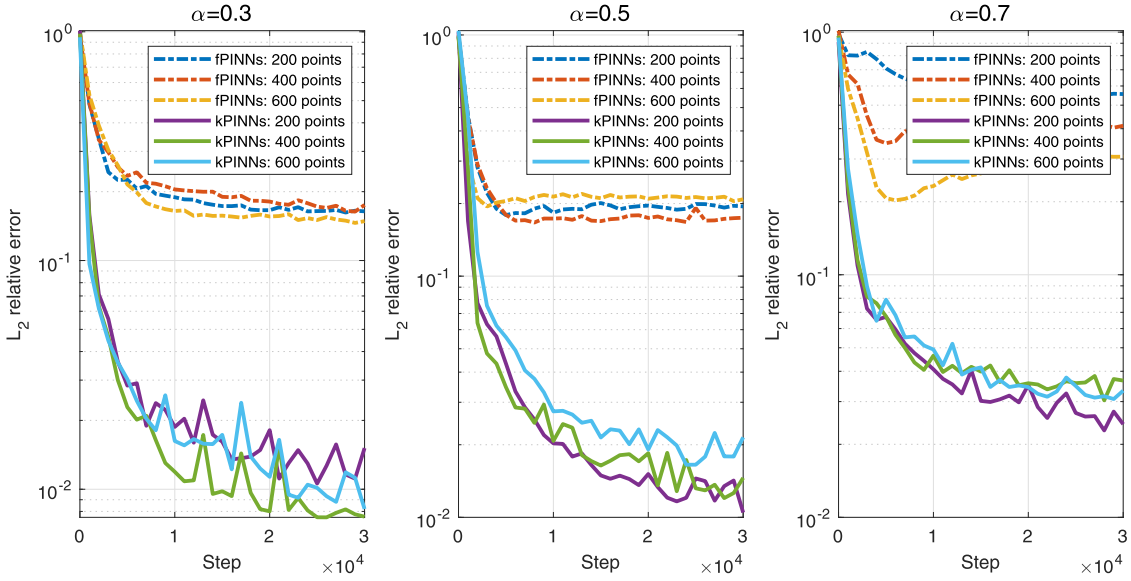


Fig. 9. L_2 relative errors of the fPINNs and kPINNs for three-dimensional problem with $\alpha = 0.3$(left), $\alpha = 0.5$(middle), $\alpha = 0.7$(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600$ for Example 4.3.

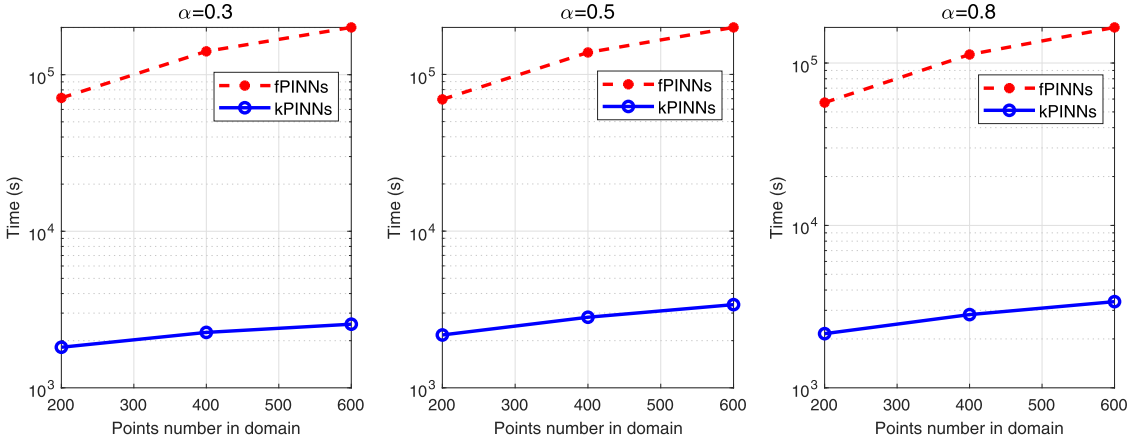


Fig. 10. Computational time of the fPINNs and kPINNs for three-dimensional problem with $\alpha = 0.3$(left), $\alpha = 0.5$(middle), $\alpha = 0.7$(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600$ for Example 4.3.

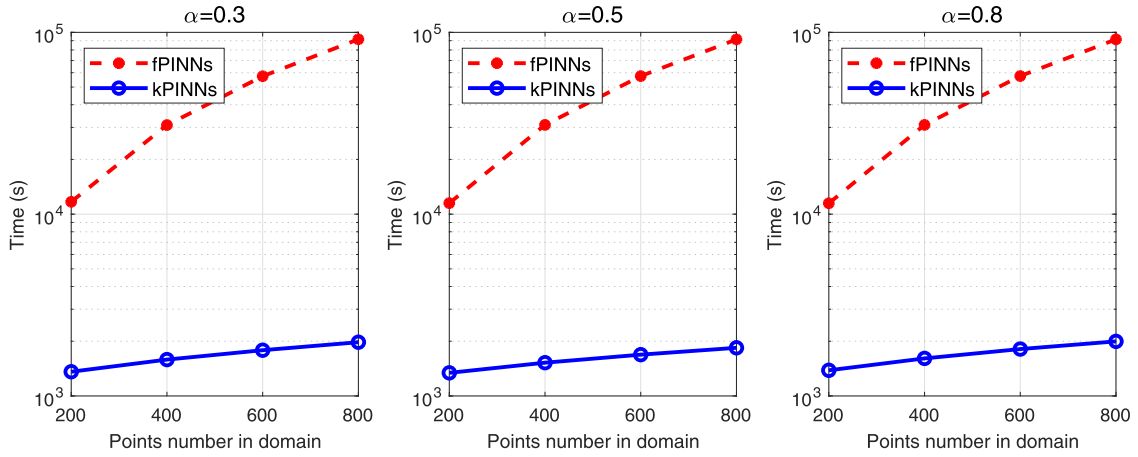


Fig. 11. Computational time of the fPINNs and kPINNs with $\alpha = 0.3$(left), 0.5(middle), 0.8(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600, 800$ for Example 4.4.

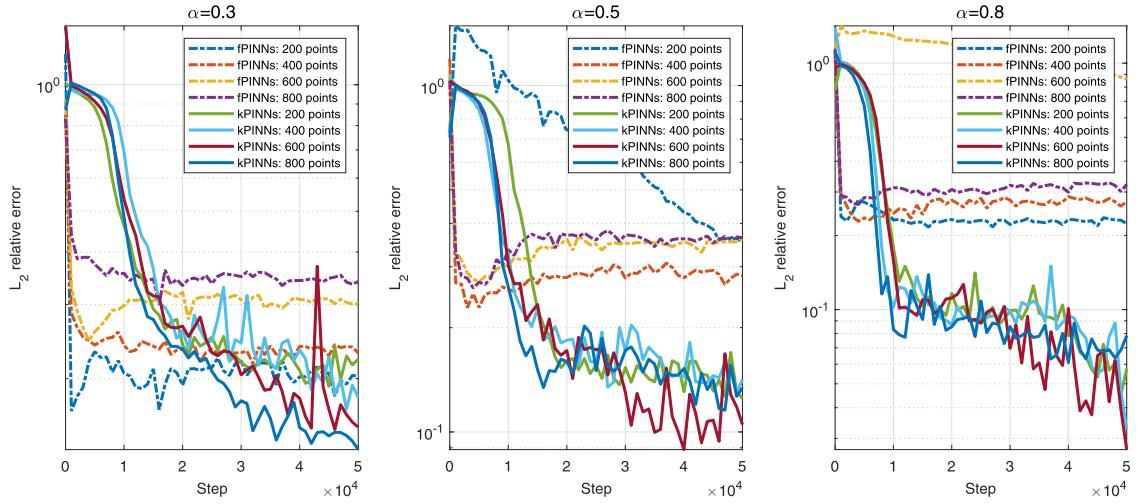


Fig. 12. L_2 relative errors of the fPINNs and kPINNs with $\alpha = 0.3$(left), 0.5(middle), 0.8(right) with $\eta = 1 \times 10^{-3}$, training steps $= 5 \times 10^4$, Adam optimizer, batch size $= M = 200, 400, 600, 800$ for Example 4.4.

with periodic boundary conditions and the exact solutions

$$u(x, y, t) = t^{\alpha} \sin(x) \sin(y).$$

The source term g is calculated by the above equation.

Accuracy tests. The training sample points are randomly chosen as $|P_f|, |P_b|, |P_{in}| = 800, 800, 500$ in each learning step from the entire space. The hidden layers are defined as $[32] * 4$. The learning rates are chosen by following Table 1. The predicted loss and L_2 relative error with $\alpha = 0.3, 0.5, 0.8$, $T = 1$ are listed in Table 6. The results show that the predicted loss and L_2 relative error of our method can reach $\mathcal{O}(10^{-4})$ and $\mathcal{O}(10^{-2})$, respectively. kPINNs can approximate the numerical solutions of the nonlinear problem with relatively less errors.

Comparisons of Computational time. Then, we define learning rate $= 1 \times 10^{-3}$, steps $= 50000$, and hidden layers as $[32] \times 3$ to calculate the computational costs and the errors of kPINNs and fPINNs with different α and different numbers of time points during model training. We define the time grid points as $t_i = i\tau$, $\tau = 1/M$, $i = 0, 1, \ldots, M$ and discretize the time fractional derivative by the L1 scheme (4.2). During model training, we choose the sample points in the domain and initial layers as $M = 200 \times k$, $k = 1, 2, 3, 4$ and 500 points on the boundary for one step. We compute the L_2 relative errors in 1000 uniform sample points at $T = 1$. We show the computational time in Fig. 11 and draw the evolution of L_2 relative errors in Fig. 12. We can observe that kPINNs give fewer errors than fPINNs. Meanwhile, the kPINNs require less computational time. The advantages become more obvious as the sample points increase.

Table 6
The predicted loss and L_2 relative error of two-dimensional problems for Example 4.4.

α	$\mathcal{L}_{pred}$	ϵ
0.3	1.80e-03	8.68e-02
0.5	8.49e-04	6.73e-02
0.8	6.65e-04	6.89e-02

Long-time simulations. We show the evolution of the solutions of time fractional Allen-Cahn equation with $g = 0, \epsilon = 0.02$ and the following initial condition [67]

$$u_0(x, y) = \tanh\Big(\frac{1}{2\epsilon} \min\{\sqrt{(x+0.3)^2 + y^2} - 0.3, \sqrt{(x-0.3)^2 + y^2} - 0.25\}\Big),$$

$x,\ y \in (-1, 1) \times (-1, 1)$.

We train the function with data-driven methods. We sample $200 \times 200 \times 500$ points at equal intervals within the domain, and choose the hidden layers to be $[32] * 4$ and the batch size to be 10000. We add a dataset $\{Y\}$ of 32,400 known numerical solutions to the model training

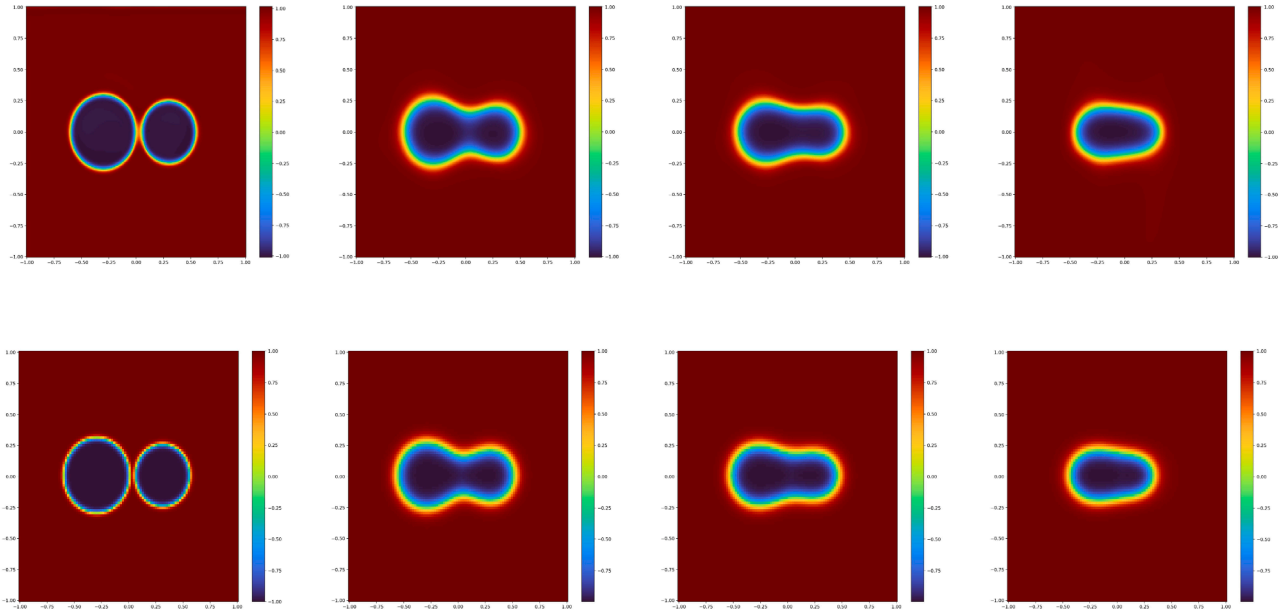


Fig. 13. The evolution of the numerical solutions by kPINNs (up) and L1 method (down) for two-dimensional problem with $\alpha = 0.8$ at $T = 0, 10, 20, 40$ for Example 4.4.

and define a new loss function as

$$\mathcal{L}_{loss} = \mathcal{L} + \lambda_{\mathcal{Y}}\mathcal{L}_{\mathcal{Y}},\ \mathcal{L}_{\mathcal{Y}} = \frac{1}{|Y|}\sum_{X\in Y}\|\hat{v}(X) - u(X)\|_2^2,$$

where $\lambda_{\mathcal{Y}} = 10$, $u(X)$ represents the known numerical solutions of X. We save the model weights with the smallest loss during model training as the optimal model. We predict the solutions with $\alpha = 0.8$ at $T = 0, 10, 20, 40$. Moreover, we use the uniform L1 scheme (4.2) in the temporal direction and the Fourier transformed method in the spatial direction to numerically solve the same problems. We present the results in Fig. 13. It can be seen that phase one (red) is undergoing a rapid transformation, evolving towards a concentration value approximating that of phase two (blue) and approaching the initial concentration value. Furthermore, compared to the evolution calculated by the traditional L1 methods, no visible difference is observed. The results indicate that the kPINNs are effective.

5. Conclusion

We have proposed kernel-based physics-informed neural networks (kPINNs) for solving a class of time-fractional differential equations and Volterra integral-differential equations. The key idea is to employ system-of-equations (SoEs) approximations within deep learning frameworks. We transform the nonlocal equations into equivalent systems of integer-order differential equations and establish both stability and error estimates for the resulting systems. Through a carefully designed loss function, the kPINNs achieve improved prediction accuracy. Numerical experiments demonstrate that the proposed kPINNs are quite effective in solving time-fractional differential equations and Volterra integral-differential equations. In the future, we will apply the operator learning to the study of non-local problems. And we will also introduce some techniques, such as the adaptive sampling strategy, to construct more efficient neural network methods for solving the related problems.

CRediT authorship contribution statement

Ruimin Gao: Writing – original draft, Software, Formal analysis, Data curation; **Lili Ju:** Writing – review & editing, Validation, Supervision, Methodology, Formal analysis; **Dongfang Li:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Formal analysis.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The computation is performed in the HPC platform of Huazhong University of Science and Technology. Codes and data that allow readers to reproduce the results in this paper are available at https://github.com/ligroup2000/ruimingao-kPINNs-2025.

Appendix A. Parameters approximation for the numerical experiments

The weights and nodes for Examples 4.1 and 4.4.

	$\alpha = 0.3$		$\alpha = 0.5$		$\alpha = 0.8$	
i	w_i	s_i	w_i	s_i	w_i	s_i
1	3.8524e-01	3.5908e-01	3.4967e-01	3.5163e-01	1.8932e-01	3.4017e-01
2	1.8809e-01	1.2894e-01	2.0735e-01	1.2364e-01	1.5259e-01	1.1572e-01
3	9.2045e-02	4.6279e-02	1.2349e-01	4.3438e-02	1.2404e-01	3.9294e-02
4	5.0064e-02	1.5892e-02	8.6633e-02	1.4209e-02	1.3083e-01	1.1883e-02
5	3.2360e-02	3.0785e-03	8.7347e-02	1.8169e-03	3.7063e-01	6.5088e-04
6	7.8903e-01	1.0000e+00	5.8968e-01	1.0000e+00	2.3488e-01	1.0000e+00
7	1.6161e+00	2.7849e+00	9.9443e-01	2.8439e+00	2.9141e-01	2.9397e+00
8	3.3099e+00	7.7555e+00	1.6770e+00	8.0878e+00	3.6155e-01	8.6418e+00
9	6.7793e+00	2.1598e+01	2.8281e+00	2.3001e+01	4.4857e-01	2.5404e+01
10	1.3885e+01	6.0148e+01	4.7692e+00	6.5412e+01	5.5653e-01	7.4681e+01
11	2.8439e+01	1.6750e+02	8.0427e+00	1.8603e+02	6.9048e-01	2.1954e+02
12	5.8247e+01	4.6647e+02	1.3563e+01	5.2904e+02	8.5667e-01	6.4538e+02

The weights and nodes for Example 4.2.

	$\alpha = 1.2$		$\alpha = 1.5$		$\alpha = 1.7$	
i	w_i	s_i	w_i	s_i	w_i	s_i
1	3.8699e-01	3.6276e-01	3.4967e-01	3.5163e-01	2.5897e-01	3.4403e-01
2	1.7195e-01	1.3159e-01	2.0735e-01	1.2364e-01	1.8804e-01	1.1836e-01
3	7.6495e-02	4.7724e-02	1.2349e-01	4.3438e-02	1.3746e-01	4.0660e-02
4	3.6423e-02	1.6846e-02	8.6633e-02	1.4209e-02	1.2563e-01	1.2655e-02
5	1.9209e-02	4.1928e-03	8.7347e-02	1.8169e-03	2.2754e-01	1.0142e-03
6	8.7098e-01	1.0000e+00	5.8968e-01	1.0000e+00	3.5668e-01	1.0000e+00
7	1.9603e+00	2.7567e+00	9.9443e-01	2.8439e+00	4.9125e-01	2.9067e+00
8	4.4119e+00	7.5993e+00	1.6770e+00	8.0878e+00	6.7658e-01	8.4491e+00
9	9.9297e+00	2.0949e+01	2.8281e+00	2.3001e+01	9.3185e-01	2.4559e+01
10	2.2348e+01	5.7749e+01	4.7692e+00	6.5412e+01	1.2834e+00	7.1388e+01
11	5.0298e+01	1.5919e+02	8.0427e+00	1.8603e+02	1.7676e+00	2.0751e+02
12	1.1320e+02	4.3885e+02	1.3563e+01	5.2904e+02	2.4345e+00	6.0317e+02
13	2.5478e+02	1.2098e+03				

The weights and nodes for Example 4.3 (Algorithm 1).

	$\alpha = 0.3$		$\alpha = 0.5$		$\alpha = 0.7$	
i	w_i	s_i	w_i	s_i	w_i	s_i
1	2.5897e-01	3.4403e-01	3.4967e-01	3.5163e-01	3.8524e-01	3.5908e-01
2	1.8804e-01	1.1836e-01	2.0735e-01	1.2364e-01	1.8809e-01	1.2894e-01
3	1.3746e-01	4.0660e-02	1.2349e-01	4.3438e-02	9.2045e-02	4.6279e-02
4	1.2563e-01	1.2655e-02	8.6633e-02	1.4209e-02	5.0064e-02	1.5892e-02
5	2.2754e-01	1.0142e-03	8.7347e-02	1.8169e-03	3.2360e-02	3.0785e-03
6	3.5668e-01	1.0000e+00	5.8968e-01	1.0000e+00	7.8903e-01	1.0000e+00
7	4.9125e-01	2.9067e+00	9.9443e-01	2.8439e+00	1.6161e+00	2.7849e+00
8	6.7658e-01	8.4491e+00	1.6770e+00	8.0878e+00	3.3099e+00	7.7555e+00
9	9.3185e-01	2.4559e+01	2.8281e+00	2.3001e+01	6.7793e+00	2.1598e+01
10	1.2834e+00	7.1388e+01	4.7692e+00	6.5412e+01	1.3885e+01	6.0148e+01

Appendix B. kPINNs for nonlocal equations.

Algorithm 1 The implementation of kPINNs.

Require: Domain $\Omega \times (0, T]$, fractional order α, SoEs approximation parameters ϵ, δ, T

1: Calculating w_i, s_i using by SoEs approximation
2: Set architecture: $[input] + [hidden\ \ layers] + [1 + N_{\mathcal{P}}]$, optimizer: Adam, learning rate η, iteration
3: Set sample collocation points: P_f (domain), P_b (boundary), P_{in} (initial)
4: Initialize network parameters θ
5: **for** epoch in range(iteration) **do**
6: Sample new batch of collocation points
7: Computer

$$\mathcal{L} = \lambda_v \mathcal{L}_v + \sum_{i=1}^{N_{\mathcal{P}}} \lambda_i \mathcal{L}_{\phi_i} + \lambda_b \mathcal{L}_b + \lambda_{in} \mathcal{L}_{in}$$

8: Update parameters: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
9: Adjust learning rate η
10: **end for**
11: Evaluate trained network on test points
12: Compute predicted error $\mathcal{L}_{pred}$ and L_2 relative error ϵ

References

[1] H. Brunner, Collocation Methods for Volterra Integral and Related Functional Differential Equations, Cambridge University Press, 2004.
[2] K. Mustapha, W. Mclean, Superconvergence of a discontinuous Galerkin method for fractional diffusion and wave equations, SIAM J. Numer. Anal. 51 (1) (2013) 491–515.
[3] Q. Du, Nonlocal Modeling, Analysis and Computation, SIAM, Philadelphia, 2019.
[4] T.C. Lin, T.Y. Lee, Chaos synchronization of uncertain fractional-order chaotic systems with time delay based on adaptive fuzzy sliding mode control, IEEE Trans. Fuzzy Syst. 19 (4) (2011) 623–635.
[5] E. Solhi, F. Mirzaee, S. Naserifar, Approximate solution of two dimensional linear and nonlinear stochastic itô-Volterra integral equations via meshless scheme, Math. Comput. Simul. 207 (2023) 369–387.
[6] Y. Rostami, K. Maleknejad, Approximate solution to solve singular variable-order fractional Volterra-Fredholm integral partial differential equations type defined using hybrid functions, Int. J. Comput. Math. 101 (6) (2024) 668–693.
[7] H. Liao, D. Li, J. Zhang, Sharp error estimate of the nonuniform L1 formula for linear reaction subdiffusion equations, SIAM J. Numer. Anal. 56 (2) (2018) 1112–1133.
[8] D. Li, J. Wang, J. Zhang, Unconditionally convergent-Galerkin FEMs for nonlinear time fractional Schrödinger equations, SIAM J. Sci. Comput. 39 (6) (2017) 3067–A3088.
[9] D. Li, J. Zhang, Efficient implementation to numerically solve the nonlinear time fractional parabolic problems on unbounded spatial domain, J Comput Phys 322 (2016) 415–428.
[10] B. Jin, R. Lazarov, Z. Zhou, Numerical methods for time fractional evolution equations with nonsmooth data: a concise overview, Comput. Methods Appl. Mech. Eng. 346 (2019) 332–358.
[11] F. Zeng, Z. Zhang, G.E. Karniadakis, Second-order numerical methods for multiterm fractional differential equations: smooth and non-smooth solutions, Comput. Methods Appl. Mech. Eng. 327 (2017) 478–502.
[12] N. Kopteva, Error analysis of an L2-type method on graded meshes for a fractional-order parabolic problem, Math. Comput. 90 (2021) 19–40.
[13] H. Liao, W. Mclean, J. Zhang, A second-order scheme with nonuniform time steps for a linear reaction-subdiffusion problem, Commun. Comput. Phys. 30 (2) (2021) 567–601.
[14] F. Mirzaee, S. Alipour, A hybrid approach of nonlinear partial mixed integro-differential equations of fractional order, Iranian J. Sci. Technol. Trans. A: Sci. 44 (2020) 725–737.
[15] Y. Rostami, K. Maleknejad, Solving nonlinear Volterra integro-differential equations in the mechanical model of inelastic cables under the influence of loads, Int. J. Numer. Modell. Electron. Networks Devices Fields 38 (5) (2025) 70115.
[16] C. Li, Z. Wang, Numerical Methods for the Time Fractional Convection-Diffusion-Reaction Equation Numerical Functional Analysis and Optimization, 42, 2021.
[17] M. Stynes, E. O'riordan, J.L. Gracia, Error analysis of a finite difference method on graded meshes for a time fractional diffusion equation, SIAM J. Numer. Anal. 55 (2017) 1057–1079.
[18] D. Li, C. Wu, Z. Zhang, Linearized Galerkin FEMs for nonlinear time fractional parabolic problems with non-smooth solutions in time direction, J. Sci. Comput. 80 (1) (2019) 403–419.
[19] Y. Rostami, Two approximated techniques for solving of system of two-dimensional partial integral differential equations with weakly singular kernels, Comput. Appl. Math. 40 (2021) 217.
[20] L. Greengard, J. Strain, A fast algorithm for the evaluation of heat potentials, Commun. Pure Appl. Math. 43 (1990) 949–963.
[21] S. Jiang, L. Greengard, Fast evaluation of nonreflecting boundary conditions for the Schrödinger equation in one dimension, Comput. Math. Appl. 47 (6–7) (2004) 955–966.
[22] X. Guo, S. Jiang, Y. Xiong, J. Zhang, Compressing the memory variables in constant-Q viscoelastic wave propagation via an improved sum-of-exponentials approximation, J. Comput. Phys. 518 (2024) 113326.
[23] S. Jiang, J. Zhang, Q. Zhang, Z. Zhang, Fast evaluation of the Caputo fractional derivative and its applications to fractional diffusion equations, Commun. Comput. Phys. 21 (3) (2017) 650–678.
[24] N. Liu, Y. Chen, J. Zhang, Y. Zhao, Unconditionally optimal H1-error estimate of a fast nonuniform $L2-1_\sigma$ scheme for nonlinear subdiffusion equations, Numer. Algorithms 92 (3) (2023) 1655–1677.
[25] W. Yuan, C. Zhang, D. Li, Linearized fast time-stepping schemes for time-space fractional Schrödinger equations, Physica D: Nonlinear Phenomena, 454 (2023) 133865.
[26] N. Samadyar, F. Mirzaee, Numerical scheme for solving singular fractional partial integro-differential equation via orthonormal Bernoulli polynomials, Int. J. Numer. Modell. Electron. Networks Devices Fields 32 (2019) 2652.
[27] Y. Rostami, K. Maleknejad, A novel approach to solving system of integral partial differential equations based on hybrid modified block-pulse functions, Math. Methods Appl. Sci. 47 (2024) 5798–5818.
[28] J. Lv, Q. Hong, X. Wang, Z. Mao, Q. Sun, Deepstsnet, Reconstructing the quantum state-resolved thermochemical nonequilibrium flowfield using deep neural operator learning with scarce data, J. Comput. Phys. 491 (2023) 112344.
[29] L. Lu, P. Jin, G. Pang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229.
[30] Z. Mao, L. Lu, O. Marxen, T.A. Zaki, G.E. Karniadakis, DeepM&Mnet for hypersonics: predicting the coupled flow and finite-rate chemistry behind a normal shock using neural-network approximation of operators, J. Comput. Phys. 447 (2021) 110698.
[31] Z. Li, H. Zheng, N. Kovachki, et al., Physics-informed neural operator for learning partial differential equations, ACM/IMS J. Data Sci. 2024 (3) (2026) 1–27.
[32] H. You, Q. Zhang, C.J. Ross, C.H. Lee, Y. Yu, Learning deep implicit fourier neural operators (IFNOs) with applications to heterogeneous material modeling, Comput. Methods Appl. Mech. Eng. 398 (2022) 115296.
[33] H. You, Y. Yu, M. D'elia, T. Gao, S. Silling, Nonlocal kernel network (NKN): a stable and resolution-independent deep neural network, J. Comput. Phys. 469 (2022) 111536.
[34] P. Schwerdtner, P. Schulze, J. Berman, B. Peherstorfer, Nonlinear embeddings for conserving Hamiltonians and other quantities with neural Galerkin schemes, SIAM J. Sci. Comput. 46 (5) (2024) 583–C607.
[35] J. Bruna, B. Peherstorfer, E. Vanden-Eijnden, Neural Galerkin schemes with active learning for high-dimensional evolution equations, J. Comput. Phys. 496 (2024) 112588.
[36] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations, Science 367 (2020) 1026–1030.
[37] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[38] L. Lu, X. Meng, Z. Mao, G. Emkarniadakis, DeepXDE: a deep learning library for solving differential equations, SIAM Rev. 63 (1) (2021) 208–228.
[39] J.D. Toscano, V. Oommen, A.J. Varghese, et al., From PINNs to PIKANs: recent advances in physics-informed machine learning, 2025.
[40] Z. Gao, L. Yan, T. Zhou, Failure-informed adaptive sampling for, PINNs SIAM J. Sci. Comput. 45 (4) (2023) 1971–A1994.
[41] G. Pang, M. D'elia, M. Parks, G.E. Karniadakis, NPINNs: nonlocal physics-informed neural networks for a parametrized nonlocal universal laplacian operator. algorithms and applications, J. Comput. Phys. 422 (2020) 109760.
[42] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of Fourier feature networks: from regression to solving multi-scale PDEs with physics-informed neural networks, Comput. Methods Appl. Mech. Eng. 384 (2021) 113938.
[43] S. Tian, Y. Yu, B. Li, A data-driven adaptive deep learning method for nonlinear Schrödinger type equation, Chin. Phys. Lett. 42 (12) (2025) 120001.
[44] S. Tian, B. Li, Z. Zhang, Data-driven Ai- and Bi-soliton of the cylindrical Korteweg-de Vries equation via prior-information physics-informed neural networks, Chin. Phys. Lett. 41 (3) (2024) 030201.
[45] K. Georgiou, C. Siettos, A.N. Yannacopoulos, Fredholm neural networks, SIAM J. Sci. Comput. 47 (4) (2025) 1006–C1031.
[46] J. Jia, X. Zhang, S. Wang, L. Ju, H. Feng, A novel deep convolutional surrogate model with incomplete solve loss for parameterized steady-state diffusion problems, J. Comput. Phys. 537 (2025) 114132.
[47] T. Lu, L. Ju, L. Zhu, A multiple transferable neural network method with domain decomposition for elliptic interface problems, J. Comput. Phys. 530 (2025) 113902.
[48] J. Huang, R. You, T. Zhou, Frequency-adaptive multi-scale deep neural networks, Comput. Methods Appl. Mech. Eng. 437 (2025) 117751.
[49] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.
[50] A.T. Deresse, T.T. Dufera, A deep learning approach: physics-informed neural networks for solving the 2D nonlinear sine-gordon equation, Results in Applied Mathematics 25 (2025) 100532.

[51] A.T. Deresse, A.S. Bekela, T.T. Dufera, MT-PINNs: Multi-term physics-informed neural networks for solving initial boundary value problems of 2D and 3D nonlinear telegraph equations, Bound. Value Probl. 2025 (2025) 146.
[52] D. Zhang, L. Guo, G.E. Karniadakis, Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks, SIAM J. Sci. Comput. 42 (2) (2020) 639–A665.
[53] A. Arzani, K.W. Cassel, R.M. Souza, Theory-guided physics-informed neural networks for boundary layer problems with singular perturbation, J. Comput. Phys. 473 (2023) 111768.
[54] R. Mattey, S. Ghosh, A novel sequential method to train physics informed neural networks for Allen Cahn and Cahn Hilliard equations, Comput. Methods Appl. Mech. Eng. 390 (2022) 114474.
[55] A.T. Deresse, A.S. Bekela, A deep learning approach: physics-informed neural networks for solving a nonlinear telegraph equation with different boundary conditions, BMC Res. Notes 18 (2025) 77.
[56] S. Wang, H. Zhang, X. Jiang, Physics-informed neural network algorithm for solving forward and inverse problems of variable-order space-fractional advection-diffusion equations, Neurocomputing 535 (2023) 64–82.
[57] G. Pang, L. Lu, G.E. Karniadakis, fPINNs: fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (4) (2019) 2603–A2626.
[58] M.D. Elia, Q. Du, C. Glusa, M. Gunzburger, X. Tian, Z. Zhou, Numerical methods for nonlocal and fractional models, Acta Numer. 29 (2020) 1–124.
[59] Z. Hu, K. Kawaguchi, Z. Zhang, G.E. Karniadakis, Tackling the curse of dimensionality in fractional and tempered fractional PDEs with physics-informed neural networks, Comput. Methods Appl. Mech. Eng. 432 (B) (2024) 117448.
[60] L. Ma, R. Li, F. Zeng, L. Guo, G.E. Karniadakis, Bi-Orthogonal fPINN: a physics-informed neural network method for solving time-dependent stochastic fractional, PDEs Commun. Comput. Phys. 34 (4) (2023) 1133–1176.
[61] B. Lu, Z. Hao, C. Moya, G. Lin, FPINN-deeponet: a physics-informed operator learning framework for multi-term time fractional mixed diffusion-wave equations, J. Comput. Phys. 538 (2026) 114184.
[62] L. Guo, H. Wu, X. Yu, T. Zhou, Monte carlo fPINNs: deep learning method for forward and inverse problems involving high-dimensional fractional partial differential equations, Comput. Methods Appl. Mech. Eng. 400 (2022) 115523.
[63] S. Wang, G.E. Karniadakis, GMC-PINNs: a new general Monte Carlo PINNs method for solving fractional partial differential equations on irregular domains, Comput. Methods Appl. Mech. Eng. 429 (2024) 117189.
[64] Halanay, On the asymptotic behavior of the solutions of an integro-differential equation, J. Math. Anal. Appl. 10 (2) (1965) 319–324.
[65] M. Zeinhofer, R. Masri, A. K, A unified framework for the error analysis of physics-informed neural networks, IMA J. Numer. Anal. 45 (5) (2025) 2988–3025.
[66] Z. Sun, X. Wu, A fully discrete difference scheme for a diffusion-wave system, Appl. Numer. Math. 56 (2) (2006) 193–209.
[67] D.C. Antonopoulou, G. Karali, D. Li, Optimal error estimates of energy-stable Runge-Kutta methods for the ϵ-dependent Cahn-Hilliard/Allen-Cahn equation, J. Sci. Comput. 103 (2025) 68.